



Maharaja Education Trust ®

MIT First Grade College

Department of Computer Applications

Industrial Suburb, Manandavadi Road, Mysuru -570008, www.mitfgc.in

Affiliated to University of Mysore, Accredited by NAAC with "A" Grade, ISO 9001:2015 Certified Institution



VISION OF THE INSTITUTE

Empower the individuals and society at large through educational excellence; sensitize them for a life dedicated to the service of fellow human beings and mother land.

MISSION OF THE INSTITUTE

To impact holistic education that enables the students to become socially responsive and useful, with roots firm on traditional and cultural values; and to hone their skills to accept challenges and respond to opportunities in a global scenario.

Lecture Notes on

Operating Systems

Prepared by

Miss. Abilasha C

Assistant Professor

Department

Computer Science – B.C.A.

OPERATING SYSTEMS

Program Name	B.C.A	Semester	II
Course Title	Operating Systems (Theory)		
Course Code:	CAM23T	No. of Credits	03
Contact hours	44	Duration of Exam	03
Formative Assessment Marks	20	Summative Assessment Marks	80

Course Outcomes (COs): After the successful completion of the course, the student will be able to:

CO1	Understand the fundamentals of the operating system.
CO2	Describe the concepts of process, process management, CPU Scheduling, process synchronization, Dead locks, memory management and Virtual Memory management.
CO3	Illustrate the file system and structure.
CO4	Understand the UNIX OS, Shell Programming, Conditional Control Structures in Shell Programming.

Unit	Contents	Hours
1	Introduction: Definition, Computer System Components, User View, System View And System Goals, Batch Systems, Multi Programmed Systems, Time-Sharing Systems, Real-Time Systems, System Components, Operating System Services. Process: Process Concept, Process State Diagram Process Control Block, Process Scheduling- Scheduling Queues, Scheduler, Cooperating Process, Interprocess Communication. CPU Scheduling: Basic Concepts, Preemptive And Non-Preemptive Scheduling, Scheduling Criteria, Scheduling Algorithms-FCFS, Shortest Job First Priority Scheduling, Round Robin Scheduling	11
2	Process Synchronization: The Critical Section Problem, Solution For Critical Section Problem, Bakery Algorithm, Semaphores-Meaning, Types Of Semaphores, Synchronization Problems- Bounded Buffer Problem, Readers-Writers Problem. Deadlocks: Deadlock Characterization, Methods For Handling Deadlocks, Deadlock Prevention, Deadlock Avoidance, Deadlock Detection, Recovery From Deadlock.	11
3	Memory Management: Introduction, Logical Versus Physical Address Space, Dynamic Loading, Dynamic Linking, Swapping, Contiguous Allocation, Partitioned Memory Allocation, Paging, Virtual Memory Management-Segmentation, Segmentation With Paging. File System: File Concepts, File Attributes, File Operations, File Types, File Structure, Access Methods, Directorystructure, File-System Structure, Allocation	11

OPERATING SYSTEMS

	Methods- Contiguous Allocation, Linked Allocation and Indexed Allocation, Free- Space Management.	
4	Introduction to Unix System: The Unix Operating System, The UNIX architecture. Shell Programming: Vi editor, shell types, shell command line processing, shell script features, executing a shell script, system and user-defined variables, expr command, shell screen interface, read and echo statement, command substitution, escape sequence characters, shell script arguments, positional parameters, test command, file test, string test, numeric test. Conditional Control Structures: if statement, case statement Looping Control Structure-while, until, for, statements. Jumping Control Structures break, continue, exit. Shell Programs covering the above concepts.	11

UNIT 1**INTRODUCTION TO OPERATING SYSTEM****DEFINITION**

An operating system is a system program which acts as an interface between the user and hardware. The purpose of an Operating system (OS) is to provide an environment in which a user can execute programs. The main goals of an OS are to make computer system convenient to the user and to use the computer hardware in an efficient manner.

COMPUTER SYSTEM COMPONENTS

An OS is an important part of computer system. In general a computer system is said to consists of following components

- Hardware
- Operating System
- Application programs
- Users

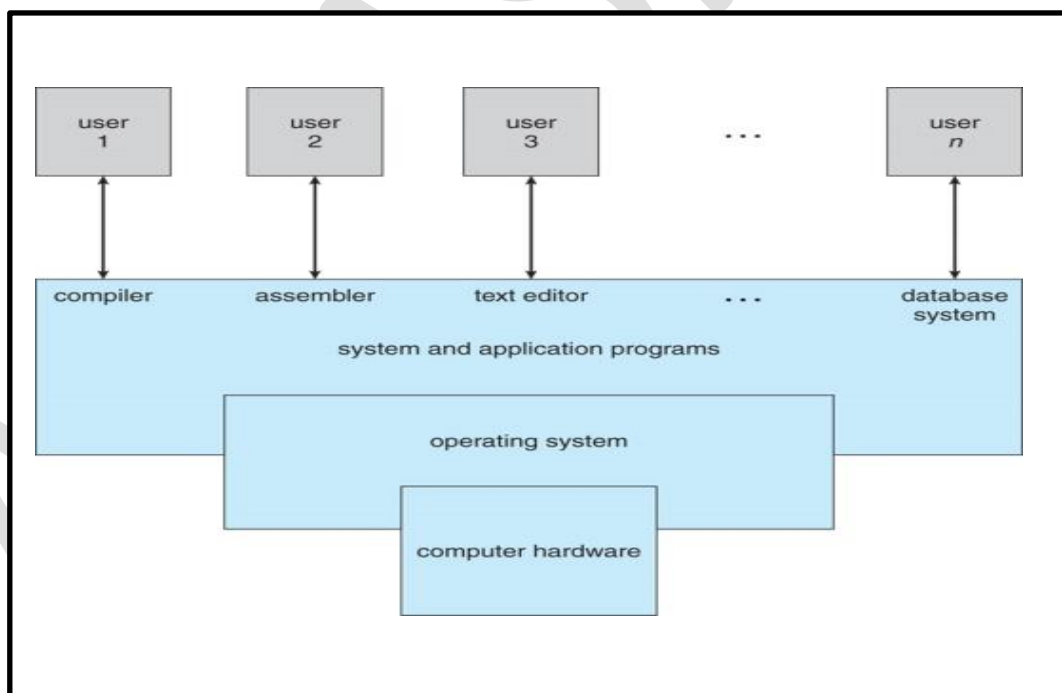


Fig 1: Abstract view of the components of a computer system

Hardware – Provides basic computing resources CPU, memory, I/O devices

Operating system - Controls and coordinates use of hardware among various applications and users.

Application programs – Defines the ways in which the system resources are used to solve the computing problems of the users ,Word processors, compilers, web browsers, database systems, video games.

Users- People, machines, other computers. There may be any number of users and any number of applications. The operating system controls and coordinates the use of the hardware among the various applications programs for the various users.

USER VIEW:

The Operating System is an interface, hides the details which must be performed and present a virtual machine to the user that makes it easier to use. Operating System provides the following services to the user.

- Execution of a program
- Access to I/O devices
- Controlled access to files
- Error detection (Hardware failures, and software errors)

SYSTEM VIEW:

Operating System is a program that functions in the same way as other programs. It is a set of instructions that are executed by the processor. Operating System acts as a program to perform the following.

- Hardware upgrades
- New services
- Fixes the issues of resources
- Controls the user and hardware operations

SYSTEM GOALS:

- Operating System is used as a communication channel between the Computer hardware and the user. It works as an intermediate between System Hardware and End-User.
- It controls all the computer resources.
- It provides valuable services to user programs.
- It coordinates the execution of user programs.
- It provides resources for user programs.
- It provides an interface (virtual machine) to the user.
- It hides the complexity of software.
- It supports multiple execution modes.
- It monitors the execution of user programs to prevent errors.

OPERATING SYSTEM SERVICES/FUNCTIONS

The aim of an operating system is to provide an environment for a user friendly and easy interaction between the computer and the user. The operating system provides certain services to programmer and users to make the programming task easier and to operate computer easily. However the services provided may differ from operating system to operating system.

1. Program Execution

The operating system must be able to load a program into memory and run it and should link to the various parts of the program properly and the program must be able to terminate normally or generate an appropriate error message if it terminates abnormally. It should also provide a proper environment to see the output of the program, which has been successfully executed.

2. Input-Output operations

A program in the state of execution may require performing an input or output operation. The operating system must be providing some means to provide the input or display the output. This may be from any device. When input-output activity is performed certain tasks such as protection control has to be given utmost importance mainly because the user cannot perform this operation directly.

3. File-System manipulation

The entire operation with the computer deals with files because it is the only means of identification. The operating system must provide facilities for creating, writing and reading a file. All files should be manipulated with their names.

4. Communication

In most situations one process has to exchange information with another process. Sometimes the processes may be generated in the same system and sometimes on different systems. Thus there is a need to have a proper and efficient means of communication. This is done by the operating system either by using shared memory or by message passing technique.

5. Error detection

Errors may occur at all stages of operations in an operating system i.e., errors may occur in the CPU during the execution of some task or during the allocation of a job in the memory, or in any hardware device, or on any user program due to various reasons, or in the input-output devices while using the system. These errors have to be taken care by operating system to ensure correct and consistent computing.

6. Resource allocator

Whether it is a single user system or multi-user system various resources have to be allocated for every program in the state of execution. Since a number of resource are managed while allocating the resources determining some appropriate policy for the resource allocation is necessary and also solving the conflicts that may occur during allocation or de-allocation of the resource are the major concerns of this service.

7. Accounting

This service tells us about what are the resources that are requested and how much of resources are used by the user for running the job. For example accounting of CPU time used, disk usage etc. By using these statistics if the system and its resources are used commercially one can be charged. Apart from that these statistics may also be useful data for researchers for improving the efficiency of the system.

8. Protection

The users who use the system to store information would like to have control over the information stored. Protection also involves securing all the resources from unauthorized access. Security may involve the user identifying him with a proper procedure and password. File protections may also be provided with either permissions or using suitable encryptions methodologies. If a system is to be protected and secure, precautions must be instituted throughout its use.

DIFFERENT TYPES OF OPERATING SYSTEM

1. Batch processing
2. Multiprogramming
3. Multi-processing or tasking
4. Time- sharing
5. Online/Real time processing

1. BATCH PROCESSING:

Batch processing is one of the oldest methods of running programs. The users in a batch operating system do not interact with the computer directly. Each user prepares its job on an off-line device like punch cards and submits it to the computer operator. To speed up the processing, jobs with similar needs are batched together and run as a group(batches). When batch of programs have been collected, the operator loads this batch of programs into the computer one at a time where they are executed one after the other. Finally, the operator retrieves the printed outputs of all these jobs and returns them to the concerned user.

When a computer is used in this way the input data (and often the program) are introduced into the computer and processed automatically, generally without operator's intervention.

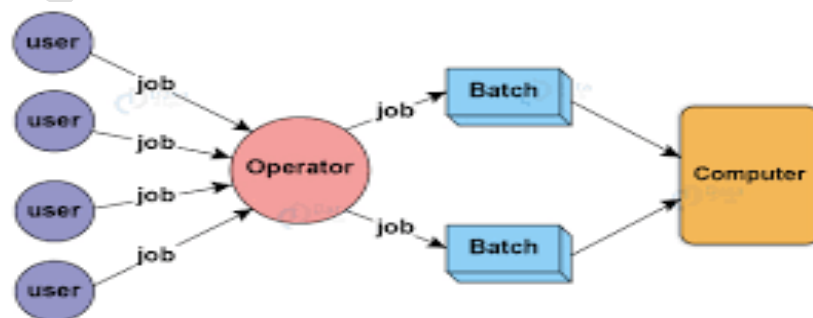


Fig 2: Batch processing operating system

Advantages

1. The method of batch processing reduces the idle time of computer system because transition from one job to another does not require operator intervention.
2. Moreover, it is most appropriate method of processing for many types of applications such as pay roll or preparation of customer statements.

Disadvantages

1. It reduces timeliness in some cases. The time required to accumulate data into batches, in some instances destroys much of the values of the data.
2. Though efficient from the computer points of view, batch processing makes each job wait in line at each step and often increases its turnaround time.
3. Lack of interaction between the user and the job.
4. CPU is often idle, because the speed of the mechanical I/O devices is slower than the CPU.

2. MULTI PROGRAMMING OPERATING SYSTEM:

In case of batch processing, a batch programs are loaded one after the in sequence into the main memory for process. Once loaded, a program will remain in the main memory until execution is completed. Thus the program which is currently being executed will be sole occupant of the user's area of main memory and it will have the CPU exclusively available to itself. When there is only one program in the main memory, the most power full resources of the systems may be underutilized.

In order to overcome the problem of under utilization of main memory and the CPU, concept of multiprogramming was introduced.

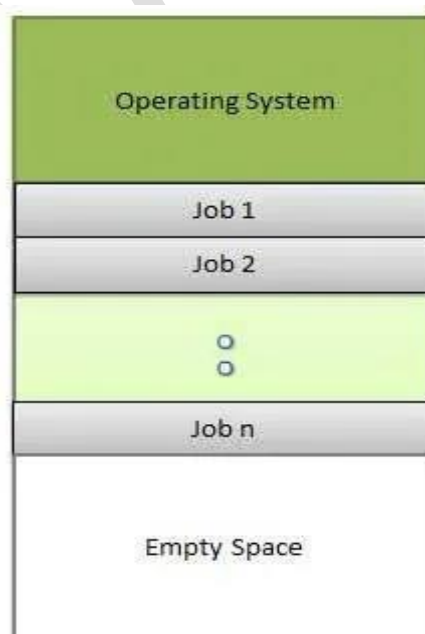


Fig 3: Multiprogramming operating system

The operating system which can run multiple processes on a single processor is called a multiprogramming operating system. In multiprogramming there will be two or more users program's in main memory and executing them concurrently.

The CPU switches from one program to another almost instantaneously. Since operating speed of CPU is faster than that of Input and Output operations. The CPU can be allocated to several programs instead of remaining idle when one is busy with I/P-O/P operations.

In multiprogramming system when one program is waiting for I/P-O/P transfer there is another program ready to utilize the CPU, thus it is possible for several users to share the time of CPU. There are numbers of programs available to the CPU. Although two or more user programs reside in the main memory simultaneously, the CPU is capable of executing a instruction at a time.

Advantages

1. CPU is used most of time and never become idle
2. The system looks fast as all the tasks runs in parallel
3. Resources are used nicely
4. Response time is shorter
5. Increased throughput & lowered response time

(Throughput: Throughput is the amount of work completed in a unit of time. In other words throughput is the processes executed to number of jobs completed in a unit of time.

Response Time: The amount of time it takes for the CPU to respond to a request made by a process.)

Disadvantages

1. **Large memory:** For multiprogramming to work satisfactorily, large main memory is required together with fast secondary storage devices like disk and fast CPU. The main memory should be large enough to accommodate a number of users programs along with the supervisor.
2. **Memory protection:** The design of computers for multiprogramming must provide memory protection mechanism to prevent a program in one memory partition from changing information or instruction of program in other memory partition.
3. **Program status preservation:** In multiprogramming, a portion of program one is executed, and a segment of another and so on. This requires stopping of a program execution and restarting its execution after sometime. In order to restart a program all the values that were stored in memory and CPU register that were being used at that time of its stopping should be restored.
4. It is highly complicated and sophisticated.
5. The CPU scheduling is required.

3. TIME SHARING OPERATING SYSTEM

Time sharing refers to the allocation of computer resources in time dependent fashion to several programs simultaneously. The principle notion of the time sharing system is to provide a large number of user direct accesses to the computer for solving problem. This is accomplished by providing a separate terminal to each user.

All these terminals are connected to the main computer system. Thus the time sharing system has many even hundreds of terminals linked up to the same computer at the same time. In time sharing the CPU time is divided among all the users on the scheduled basis. The basic idea behind time sharing systems is to allow all user programs to have a brief share of the CPU time. Each user program from the first program till the last is allocated a very short period of CPU time. This short period of time during which a user gets the attention of the CPU is known as **time slice**, **time slot** or **quantum** and is typically of order of 10-20 ms.

Even though it may appear that several users are using the computer system at the same time a single CPU can only execute one instruction at a time. Thus like a multiprogramming system even with a time sharing system only one program can be in control of the CPU at any given time.

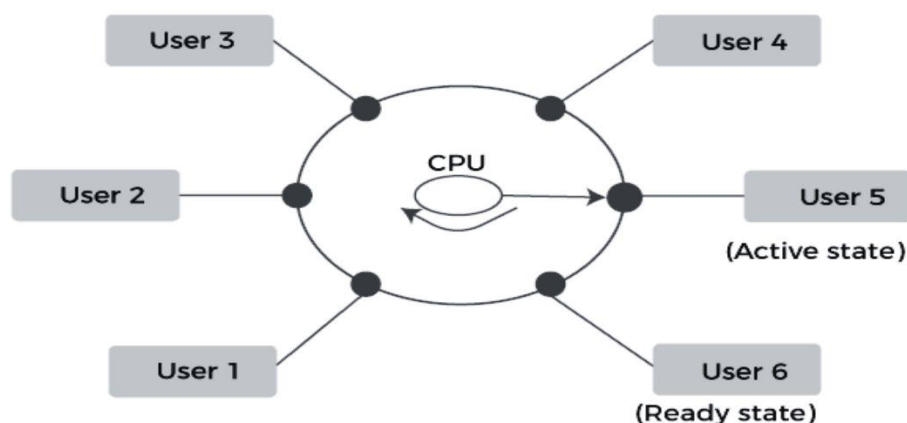


Fig 4: Time sharing operating system

Advantages

1. Reduces CPU idle time.
2. Offers computing facility to smaller user. Smaller user can gain direct access to much more sophisticated hardware and software than they could otherwise justify or afford.
3. Provides quick response and the turnaround time or the response time is negligible in case of timesharing system.

Disadvantages

1. Question of security: Since hundreds of user use a time sharing system simultaneously provision must be made to provide the security and integrity to user programs and data.
2. Problem of reliability: A time sharing system should be highly reliable as it centers to the needs of several users. Hence provision must be made to provide dependable and continuous services.

3. Problem of data communication: In a time sharing system interaction with the main computer system through remote terminals requires data communication facilities.

4. REAL TIME PROCESSING

These types of OSs serve real-time systems. The time interval required to process and respond to inputs is very small. This time interval is called **response time**.

Real-time systems are used when there are time requirements that are very strict like missile systems, air traffic control systems, robots, etc.

Two types of Real-Time Operating System which are as follows:

a. Hard Real-Time Operating system:

A hard real-time operating system is used when we need to complete tasks by a given deadline. If the task is not completed on time then the system is considered to be failed.

For example: The pacemaker. A pacemaker has flexible, insulated wires (leads) that are placed in one or more chambers of the heart. These wires deliver electrical pulses to adjust the heart rate. And these wires require a hard real-time operating system. These systems are built for saving life like automatic parachutes or airbags which are required to be readily available in case of any accident. Virtual memory is rarely found in these systems.

b. Soft Real-Time Operating System

A soft real-time operating system is used where few delays in time duration are acceptable. That is if the given task is taking a few seconds more than the specified time then also no critical damage takes place. **For example,** telephone switches, the sending or receiving of the call can take some time. It will not be considered a failure.

Advantages

1. **Maximum Consumption:** Maximum utilization of devices and system, thus more output from all the resources
2. **Task Shifting:** The time assigned for shifting tasks in these systems are very less. For example, in older systems, it takes about 10 microseconds in shifting one task to another, and in the latest systems, it takes 3 microseconds.
3. **Focus on Application:** Focus on running applications and less importance to applications which are in the queue.
4. **Real-time operating system in the embedded system:** Since the size of programs are small, RTOS can also be used in embedded systems like in transport and others.
5. **Error Free:** These types of systems are error-free.
6. **Memory Allocation:** Memory allocation is best managed in these types of systems.

Disadvantages

1. **Limited Tasks:** Very few tasks run at the same time and their concentration is very less on few applications to avoid errors.
2. **Use heavy system resources:** Sometimes the system resources are not so good and they are expensive as well.
3. **Complex Algorithms:** The algorithms are very complex and difficult for the designer to write on.
4. **Device driver and interrupt signals:** It needs specific device drivers and interrupts signals to respond earliest to interrupts.
5. **Thread Priority:** It is not good to set thread priority as these systems are very less prone to switching tasks.

Some examples of these fields are Airline traffic control systems, Command Control Systems, airline reservation systems, Heart Pacemaker, Network Multimedia Systems, etc.

PROCESS MANAGEMENT

DEFINITION

Process is a program in execution. The process has been given many definitions for instance.

1. An asynchronous activity.
2. The entity to which processors are assigned.
3. The 'dispatch able' unit.

But the definition "program in execution" seems to be most frequently used.

PROCESS CONCEPT

A process is basically a program in execution. The execution of a process must progress in a sequential fashion. The relation between process and program is, if it is not executing it is called **program** and when it is executing becomes **process**. Process, on the other hand, includes:

1. Current value of program counters (PC).
2. Contents of the processors register
3. Value of the variables
4. The process stack(SP) which typically contains temporary data such as subroutine parameter, return address, and temporary variables
5. A data section that contains global variables

PROCESS STATE

A process state changes while executing. The process states are as follows

1. **New** – the process is being created.
2. **Running** – in this state the instructions are being executed.
3. **Waiting** – the process is in waiting state until an event occurs like I/O operation completion or receiving a signal. A process moves from run state to block or wait state if it requires an I/O operation or some blocked resource during its execution. After the I/O operation gets completed or resource becomes available, the process moves to the ready state.
4. **Ready** – A process moves from new state to ready state after it is loaded into the main memory and is ready for execution. In ready state, the process waits for its execution by the processor.
5. **Terminated** – the process has finished execution.

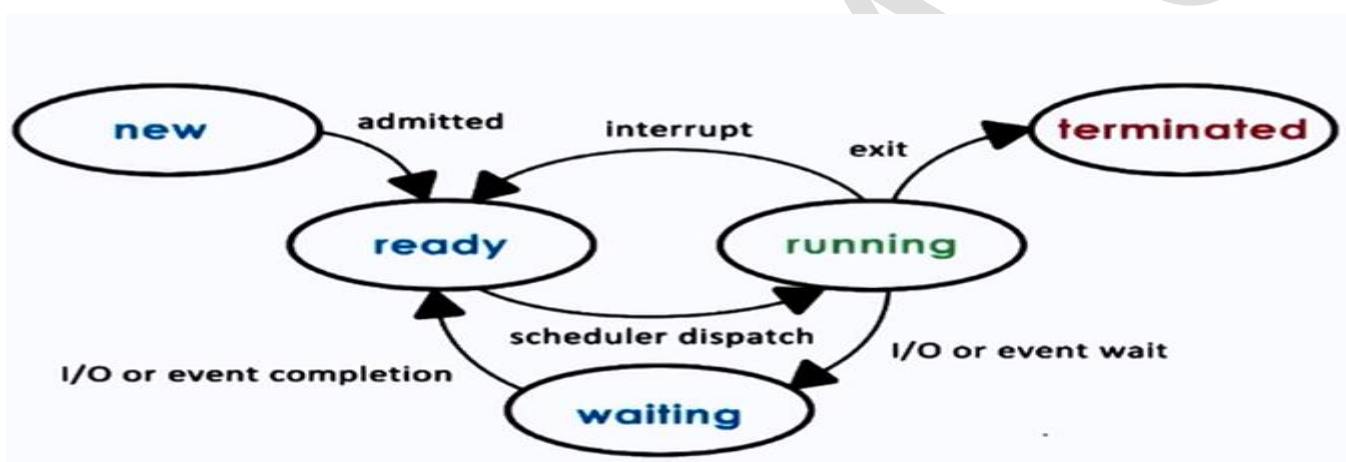


Fig 5: Process management in operating system.

PROCESS CONTROL BLOCK

A process in an operating system is represented by a data structure known as process control block (PCB) or process descriptor or task control block.



Fig 6: Process management in operating system.

The following are the data items –

- **Process State:** This specifies the process state i.e. new, ready, running, waiting or terminated.
- **Process Number:** This shows the number of the particular process.
- **Program Counter:** This contains the address of the next instruction that needs to be executed in the process.
- **Registers:** This specifies the registers that are used by the process. They may include accumulators, index registers, stack pointers, general purpose registers etc.
- **List of Open Files:** These are the different files that are associated with the process

CPU Scheduling Information: The process priority, pointers to scheduling queues etc. is the CPU scheduling information that is contained in the PCB. This may also include any other scheduling parameters.

Memory Management Information: The memory management information includes the page tables or the segment tables depending on the memory system used. It also contains the value of the base registers, limit registers etc.

I/O Status Information: This information includes the list of I/O devices used by the process, the list of files etc.

Accounting information: The time limits, account numbers, amount of CPU used, process numbers etc. are all a part of the PCB accounting information.

PROCESS SCHEDULING

CPU scheduling is the basis of multiprogrammed operating system. By switching the CPU among the process, the operating system can make the computer more productive.

Basic Concepts:

The objective of multiprogramming is to have some process running at all times, in order to maximize CPU utilization. In a uniprocessor system, only one process may run at a time, any other process must wait until the CPU is free and can be rescheduled.

The idea of multiprogramming is relatively simple. In multiprogramming several process are kept in the memory at all time when one process has to wait for some event the OS takes the CPU away from the process and gives the CPU to another process. This process continues.

Scheduling is the fundamental function of OS. Almost all computer resources are scheduled before use. The CPU is one of the primary computer resources so CPU scheduling is central to OS.

CPU - I/O BURST CYCLE

Process execution consists of a cycle of CPU execution and I/O wait. Processes alternate between these two states. Process execution begins with a CPU burst that is followed by an I/O burst, which is followed by

another CPU burst, then another I/O burst, and so on. Eventually, the final CPU burst ends with a system request to terminate execution.

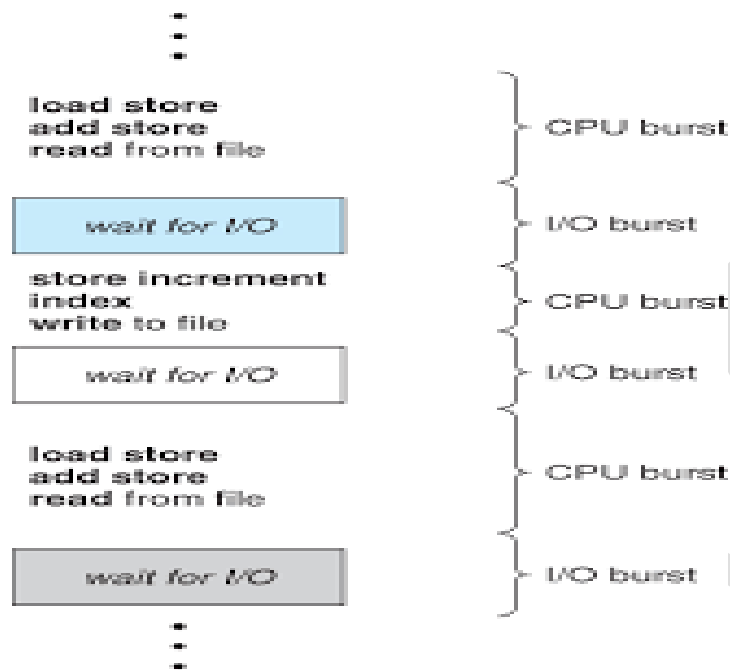


Fig 7: Alternate sequence of CPU and I/O Burst.

SCHEDULING QUEUES

Job Queue-As soon as any process enters the system, the system assembles it in the job queue. The system stores the job queue on the mass storage device i.e. secondary memory.

Ready Queue-if the process is ready for execution, Os brings it to the ready queue i.e. in the main memory. The process in the ready queue is the one those are waiting to get the CPU cycle.

Device Queue-The operating system maintains a separate device queue for each I/O device. This device queue holds the processes that are waiting to perform I/O on the device.

A new process is initially put in the ready queue. It waits in the ready queue until it is selected for execution or dispatched. Once the process is assigned to the CPU and is executing one of the events could occur.

- The process could issue an I/O request and then be placed in an I/O queue.
- The process could create a new sub process and wait for its termination.
- The process could be removed forcibly from the CPU, as a result of an interrupt and be put back in the ready queue.

In the first two cases, the process eventually switches from the waiting state to the ready state and it is then put back in the ready queue. A process continues this cycle until it terminates, at which time it removed from all queues and has its PCB and resources delocated.

Each rectangular box represents a queue. Two types of queues are present: the ready queue and device queues. The circle represents the resources they serve the queues and the arrows indicates the flow of process in the system.

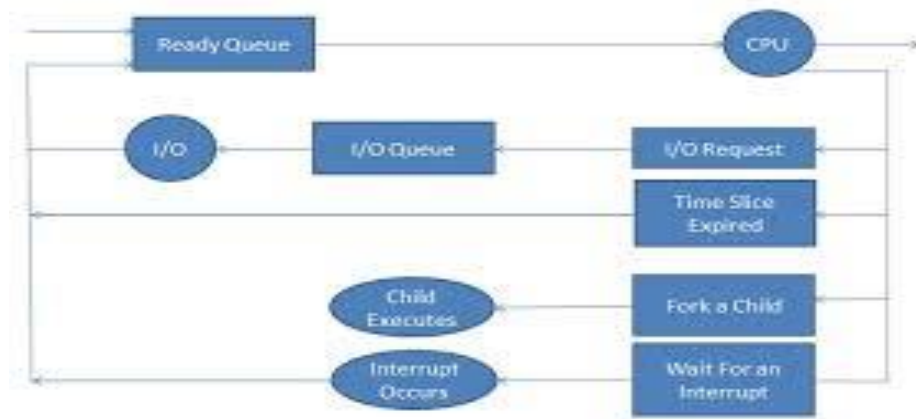


Fig 8: A common representation of process scheduling is queuing diagram.

SCHEDULERS

Schedulers are a part of the OS which handles the scheduling task they are

1. Long term scheduler
2. Short term scheduler
3. Medium term scheduler

1. Long Term Scheduler

The job scheduler or long-term scheduler selects processes from the mass storage device (typically secondary storage like disk) and loads them into the ready queue in the main memory for execution.

The long-term scheduler controls the degree of multiprogramming. The job of the long-term scheduler is very important and directly affects the system for a long time.

2. Short Term Scheduler

The short-term scheduler or CPU scheduler selects one of the processes from the ready queue and schedules them for execution. The short-term scheduler executes much more frequently than the long-term scheduler as a process may execute only for a few milliseconds.

3. Medium Term Scheduler

This scheduler removes the processes from memory (and from active contention for the CPU), and thus reduces the degree of multiprogramming.

A running process may become suspended if it makes an I/O request. Suspended processes cannot make any progress towards completion. In this condition, to remove the process from memory and make space for other processes. At some later time, the process can be reintroduced into memory and its execution can be

continued where it left off. This scheme is called **swapping**. The process is swapped out, and is later swapped in, by the medium term scheduler.

PROCESS OPERATIONS

1. Process Creation

A process can create several new processes through **create process system call** during the process execution. The process which creates a another process is called as parent process and the new process is a child process.

Every new process creates another process forming a tree-like structure. It can be identified with a unique process identifier that usually represents it as Pid which is typically an integer number. Every process needs some resources like CPU time, memory, file, I/O devices to accomplish. Whenever a process creates a sub process, and may be each sub process is able to obtain its resources directly from the operating system or from the resources of the parent process. The parent process needs to partition its resources among all its children or it may be able to share some resources to several children.

Restricting a child process to a subset of the parent's resources prevents any process from overloading the system by creating too many sub-processes. A process is going to obtain its resources whenever it is created.



Fig 9: A tree of process on a typical Solaris system.

Whenever a process creates a new process, there are two possibilities in terms of execution, which are as follows –

- The parent continues to execute concurrently with its children.
- The parent waits till some or all its children have terminated.

There are two more possibilities in terms of address space of the new process, which are as follows –

- The child process is a duplicate of the parent process.
- The child process has a new program loaded into it.

2. Process Termination

A process terminates when it finishes executing its last statement and request the operating system to delete it by issuing the “**exit system call**”. And the process may return data, output to its parent process when it exits using fork system call. And when a process exits all the resources of that process including physical, virtual memory, open files, and input/output buffers are de - allocated by the operating system.

Reasons for process termination

The reasons that the process may terminate the execution of one of its children are as follows –

- The child exceeds its usage of resources that it has been allocated.
- The task that is assigned to the child is no longer required.
- The parent is exiting and the operating system does not allow a child to continue if its parent terminates.

INTER PROCESS COMMUNICATION [IPC]

Interprocess communication is communication between two processes that may be on the same system or on a different system. The process in the operating system is of two types:

1. Independent Process
2. Cooperating Process

An **Independent process** is one that doesn't get affected by the other processes running in the system. It doesn't even affect the processing of any other process in the system. This is because the independent process doesn't share any data with the other process.

The **cooperating process** is one that shares data with the other processes in the system. Thus, it can get affected by the other processes of the system. And even it can affect the working of other processes. So, among these two processes, the cooperating process requires interprocess communication.

Methods of Interprocess Communication

There are two methods or two models that operating systems can implement to achieve IPC. Some operating systems implement both of these models.

1. Shared Memory
2. Message Passing

1. Shared Memory System

In shared memory system a cooperating process creates a shared memory segment in its address space. Now, if there occurs another process that wants to communicate with this process. Then it must attach itself to the created shared memory segment. The processes share data by reading and writing the data in the shared segment of the processes.

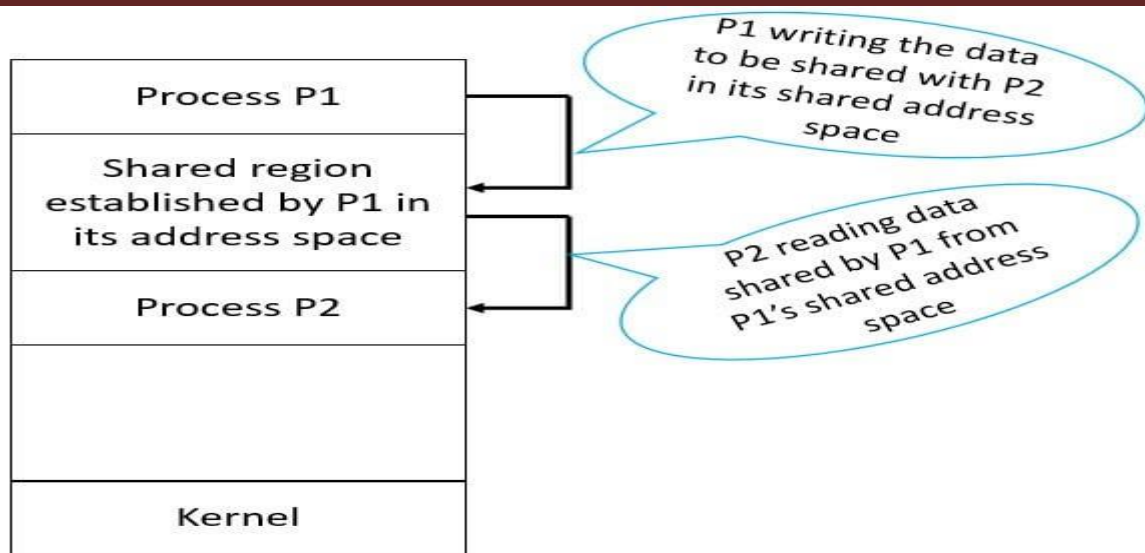


Fig 10: Shared Memory System.

Consider two cooperating processes P1 and P2. Both the processes P1 and P2 have their different address spaces. Now let us assume, P1 wants to share some data with P2.

So, P1 and P2 will have to perform the following steps:

Step 1 – Process P1 has some data to share with process P2. First P1 takes initiative and establishes a shared memory region in its own address space and stores the data or information to be shared in its shared memory region.

Step 2 – Now, P2 requires the information stored in the shared segment of P1. So, process P2 needs to attach itself to the shared address space of P1. Now, P2 can read out the data from there.

Step 3 – The two processes can exchange information by reading and writing data in the shared segment of the process.

2. Message Passing System

Message Passing provides a mechanism to allow processes to communicate and to synchronize their actions without sharing the same address space. An IPC facility generally provides two operations

- 1) Send
- 2) Receive

Message sent by processes can be either fixed or variable size. In order to send and receive message from each other, the processes must have a communication link existing between them. The link can be implemented in many ways.

Logically a link can be implemented using the following methods:

1. Direct or Indirect
2. Synchronous and Asynchronous Communication
3. Automatic and Explicit Buffering

For processes which want to communicate there must be a way in order to refer to each other. They can be either direct or indirect communication.

1. Direct Communication:

In this method each processes which wants to communicate must explicitly name the sender or recipient of the communication.

In this method the send and receive primitives are defined as follows

Send (P, message) – means send a message to process P

Receive (Q, message) – means receive a message from process Q

Addressing in this scheme is **symmetric**, that is both receiver and sender processes have to name each other to communicate.

In **asymmetric** type, only the sender names the recipient, but recipient need not name the sender so, the send and receive primitive in this scheme is defined as,

Send (P, message) – means send a message to process P

Receive (id, message) – means receives message from any process

Where the variable id is set to the process name with which communication takes place.

2. Indirect Communication:

In this scheme, the sending and receiving of message is done using a mail box (called as ports). A mail box abstractly can be viewed as an object, where the processes place their message and removes the messages from it. Each mail box will have a unique identification and a process can communicate with some other process, through number of different mailbox. Communication between two processes is possible, only if they have a shared mailbox.

The send, receive primitives are defined as,

Send (B, message) – means send a message to mailbox B

Receive (B, message) – means receive a message from mailbox B

Synchronous and Asynchronous Communication

Communication happens using send() and receive(). There are many options for these two primitives. Message passing may be blocking or non-blocking also known as synchronous and asynchronous.

- **Blocking send** – sending is blocked, until a message is received by receiving process or mailbox.
- **Non-blocking send** – sending process sends the message and resumes operation.
- **Blocking receive** – receiver blocks until a message is available.
- **Non-blocking receive** – the receiver retrieves either a valid message or a null.

3 Automatic and Explicit Buffering

The number of messages that can be present is determined by a link and is considered as queue of messages to the link. This queue can be implemented using the following 3 methods

1) Zero Capacity Queue:

This has maximum length zero, so a link cannot maintain any message waiting in it. Thus sender must wait until the recipient receives the message. And also the two processes communicating must be synchronized for a message transfer to happen. This queue type is also called as message system with no buffering.

2) Bounded Capacity Queue:

This type will have a finite length say 'n'. So at the most 'n' messages can be stored in it. The new messages sent can be placed in queue, if it is not full. And sender can continue execution. When the link is full, the sender must wait, until the queue space becomes free. This queue provides automatic buffering.

3) Unbounded Capacity Queue:

This has a infinite length and hence any number of messages can wait in it. So, the sender will not be delayed anytime. This queue type also provides automatic buffering.

CPU SCHEDULING

It is a process of determining which process will own CPU for execution while another process is on hold. The main task of CPU scheduling is to make sure that whenever the CPU remains idle, the OS at least select one of the processes available in the ready queue for execution. The selection process will be carried out by the CPU scheduler. It selects one of the processes in memory that are ready for execution.

BASIC CONCEPTS

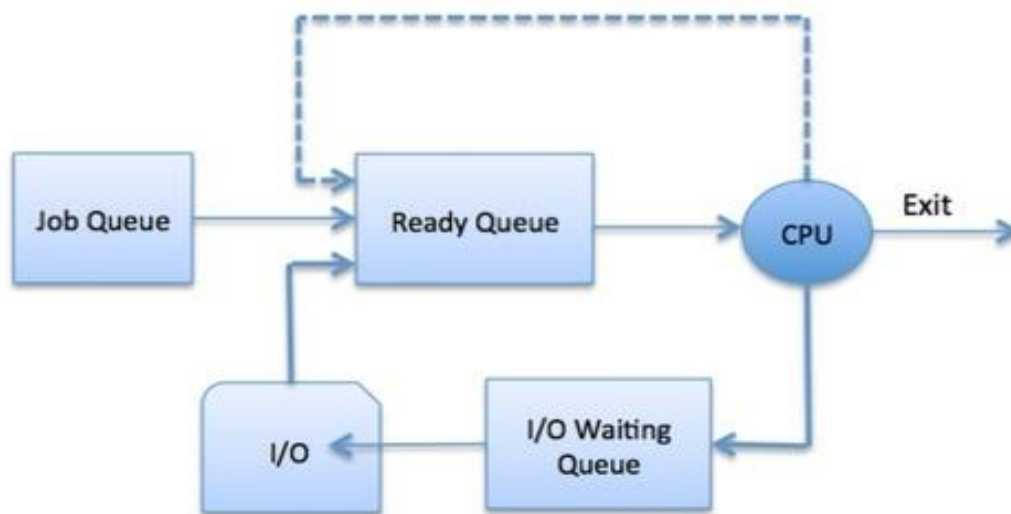


Fig 11: CPU Scheduling.

TYPES OF SCHEDULING

There are two categories of scheduling:

- **Preemptive**
- **Non-Preemptive Scheduling.**

1. PREEMPTIVE SCHEDULING

In preemptive scheduling, the tasks are assigned based on their priorities. For example, a low-priority task is running. After some time, a high-priority task comes into the system for execution, and then since the new task is of high priority, the operating system will keep the low priority task on hold and allocate the CPU to the high priority task. As the high-priority task will finish its execution, the CPU will be reallocated to the low-priority task. This is known as preemptive scheduling.

Preemptive scheduling also occurs when a process switches from the running state to the ready state and switches from the waiting state to the ready state.

2. NON-PREEMPTIVE SCHEDULING

In non-preemptive scheduling, once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU either by terminating or by switching to the waiting state. New processes are executed only after the current process has completed its execution. The process holds the resources of the CPU (CPU time) till its state changes to terminated or is pushed to the process waiting state. If a process is currently being executed by the CPU, it is not interrupted till it is completed.

Once the process has completed its execution, the processor picks the next process from the ready queue **(the queue in which all processes that are ready for execution are stored).**

SCHEDULING CRITERIA

- **CPU Utilization:** To make out the best use of the CPU and not to waste any CPU cycle, the CPU would be working most of the time (Ideally 100% of the time). Considering a real system, CPU usage should range from 40% (lightly loaded) to 90% (heavily loaded.)
- **Throughput:** It is a measure of the jobs completed within given time interval.
Throughput = $\frac{\text{the number of jobs completed}}{\text{Time interval.}}$
- **Turnaround time:** It is defined as the time interval between submission times of a process to the time of completion.
(The turnaround time is the sum of periods spent waiting to get into memory, waiting in the ready queue, executing on the CPU, and doing I/O.)

- **Waiting time:** The amount of time that a process spends waiting in the ready queue, waiting to access the control of CPU.
- **Response time:** It is a measure of the time interval between jobs submission and the first response from the system. Actually, it is the time till the first response and not the completion of process execution (final response).

SCHEDULING ALGORITHMS

CPU scheduling deals with the problem of deciding which of the process in the ready queue is to be allocated to the CPU. There are many CPU scheduling algorithms. The characteristics that are used for comparison can make a substantial difference in the determination of best algorithm.

1. First come first serve scheduling (FCFS)

In the "First come first serve" scheduling algorithm, as the name suggests, the process which arrives first, gets executed first, or we can say that the process which requests the CPU first, gets the CPU allocated first.

- First Come First Serve is just like **FIFO** (First in First out) Queue data structure, where the data element which is added to the queue first, is the one who leaves the queue first.
- This is used in Batch Systems.
- It's **easy to understand and implement** programmatically.
- A perfect real life example of FCFS scheduling is **buying tickets at ticket counter**.

Example: Consider the processes P1, P2, P3, P4 given in the below table, arrives for execution in the same order, with **Arrival Time 0**, and given **Burst Time**, let's find the average waiting time using the FCFS scheduling algorithm.

PROCESS	BURST TIME
P1	21
P2	3
P3	6
P4	2



The average waiting time will be = $(0 + 21 + 24 + 30) / 4 = 18.75$ ms



This is the GANTT chart for the above processes

The average waiting time will be **18.75 ms**

The average turnaround time will be $(21+24+30+32) / 4 = 26.75$ ms

For the above given processes, first **P1** will be provided with the CPU resources,

Hence, waiting time for **P1** will be 0

- **P1** requires 21 ms for completion, hence waiting time for **P2** will be 21 ms
- Similarly, waiting time for process **P3** will be execution time of **P1** + execution time for **P2**, which will be $(21 + 3)$ ms = 24 ms.
- For process **P4** it will be the sum of execution times of **P1**, **P2** and **P3**.

Advantages

- Easy to implement.
- First come, first serve method

Disadvantages

The average waiting time is much higher than the other algorithms.

- FCFS is very simple and easy to implement and hence not much efficient.
- FCFS scheme is not useful in scheduling interactive users because it cannot guarantee good response time.
- One of the major drawbacks of this scheme is that the average time is often quite long.
- The First-Come-First-Served algorithm is rarely used as a master scheme in modern Operating systems but it is often embedded within other schemes.

2. Shortest Job First (SJF)

Shortest-Job-First (SJF) is a scheduling algorithm in which waiting process with Smallest CPU burst (estimated run-time-to-complete) is run next. In other words, when CPU is available, it is assigned to the process that has smallest next CPU burst. If two or more processes having the same CPU burst time, they are served in FCFS order.

SJF algorithm can be implemented using both non-preemptive and preemptive methods.

i. Non Pre-emptive Shortest Job First

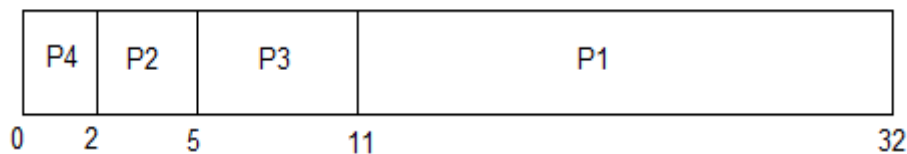
Example: Consider the below processes available in the ready queue for execution, with arrival time as 0 for all and given burst times

PROCESS	BURST TIME
P1	21
P2	3
P3	6
P4	2



In Shortest Job First Scheduling, the shortest Process is executed first.

Hence the GANTT chart will be following :



The average waiting time will be = $(0+2+5+11)/4 = 4.5\text{ms}$

The Average turnaround time will be = $(2+5+11+32)/4 = 12.5\text{ms}$

As you can see in the **Gantt chart** above, the process **P4** will be picked up first as it has the shortest burst time, then **P2**, followed by **P3** and at last **P1**.

Problem with Non Pre-emptive SJF

If the **arrival time** for processes are different, which means all the processes are not available in the ready queue at time 0, and some jobs arrive after some time, in such situation, sometimes process with short burst time have to wait for the current process's execution to finish, because in Non Pre-emptive SJF, on arrival of a process with short duration, the existing job/process's execution is not halted/stopped to execute the short job first.

Advantages

- The SJF scheduling is especially appropriate for batch processes for which the run times are known in advance.
- Since the SJF scheduling algorithm gives the minimum average time for a given set of processes, it is probably optimal.

Disadvantages

- The SJF algorithm favors short jobs (or processors) at the expense of longer ones.

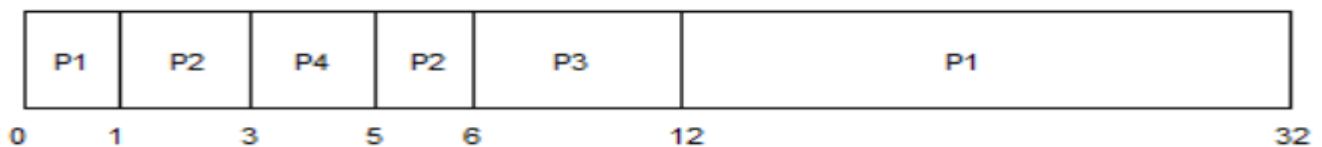
ii. Preemptive Shortest Job First

In this, jobs are moved into the ready queue when they arrive. Those Processes which have less burst time begins its execution first. When the process with less burst time arrives, then the current process stops execution, and the process having less burst time is allocated with the CPU first.

Example: Consider the processes P1, P2, P3, and P4. The arrival time and burst time of the processes are given in the following table.

PROCESS	BURST TIME	ARRIVAL TIME
P1	21	0
P2	3	1
P3	6	2
P4	2	3

The GANTT chart for Preemptive Shortest Job First Scheduling will be,



The average waiting time will be, $((5-3) + (6-2) + (12-1))/4 = \underline{4.25 \text{ ms}}$

3. Round Robin Scheduling Algorithm

Round Robin (RR) scheduling algorithm is mainly designed for time-sharing systems. This algorithm is similar to FCFS scheduling, but in Round Robin(RR) scheduling, preemption is added which enables the system to switch between processes.

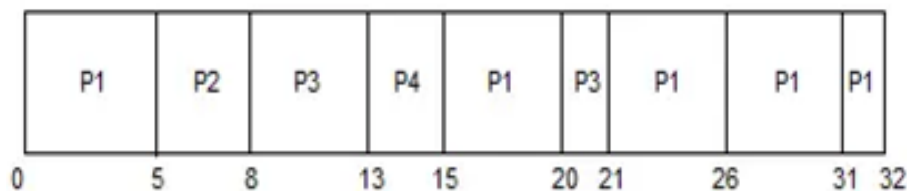
- A fixed time is allotted to each process, called a **time slice or time slot or quantum**, for execution.
- Once a process is executed for the given time period that process is preempted and another process executes for the given time period.
- So Round Robin Scheduling algorithm works as Preemptive Algorithms.
- Context switching is used to save states of preempted processes.
- It is important to note here that the length of time quantum is generally from 10 to 100 milliseconds in length.

Example: Let us now cover an example for the same

PROCESS	BURST TIME
P1	21
P2	3
P3	6
P4	2



The GANTT chart for round robin scheduling will be,



Note: In the above diagram, arrival time is not mentioned so it is taken as 0 for all processes.

Consider the value of time quantum = 5ms

Let us now calculate the Turnaround time and waiting time for the above example

Turn Around Time = Completion Time – Arrival Time

Waiting Time = Turn Around Time – Burst Time

Average waiting time = $(11 + 5 + 15 + 13) / 4 = 44 / 4 = 11\text{ms}$

Average turnaround time = $(32 + 8 + 21 + 15) / 4 = 76 / 4 = 19\text{ms}$

Advantages

- In terms of average response time, this algorithm gives the best performance.
- With the help of this algorithm, all the jobs get a fair allocation of CPU.
- In this algorithm, there are no issues of starvation.
- This algorithm deals with all processes without any priority.
- In this scheduling algorithm, each process gets a chance to reschedule after a particular quantum time.

Disadvantages

- This algorithm spends more time on context switches.
- For small quantum, it is time-consuming scheduling.
- This algorithm offers a larger waiting time and response time.
- In this, there is low throughput.
- If time quantum is less for scheduling then its Gantt chart seems to be too big.

4. Priority CPU Scheduling

In the Shortest Job first scheduling algorithm, the priority of a process is generally the inverse of the CPU burst time, i.e. the larger the burst time the lower is the priority of that process. In case of priority scheduling the priority is not always set as the inverse of the CPU burst time, rather it can be internally or externally set, but yes the scheduling is done on the basis of priority of the process where the process which is most urgent is processed first, followed by the ones with lesser priority in order. Processes with same priority are executed in FCFS manner.

The priority of process, when internally defined, can be decided based on **memory requirements, time limits, number of open files, ratio of I/O burst to CPU burst** etc. Whereas, external priorities are set based on criteria outside the operating system, like the importance of the process, funds paid for the computer resource use, market factor etc.

Priority scheduling can be of two types:

1. Preemptive Priority Scheduling.

2. Non-Preemptive Priority Scheduling.

1. Preemptive Priority Scheduling

If the new process arrived at the ready queue has a higher priority than the currently running process, the CPU is preempted, which means the processing of the current process is stopped and the incoming new process with higher priority gets the CPU for its execution.

Example: (Preemptive Priority scheduling)

Here we have taken four processes, P0, P1, P2, and P3, to be executed using Preemptive scheduling as a priority scheduling algorithm (**Lower the Number, Higher the priority**).

Process	Arrival Time	Priority	CPU Burst Time
P0	0	4	5
P1	1	1	2
P2	3	3	4
P3	4	2	3

Gantt chart:

P0	P1	P2	P3	P2	P0	
0	1	3	4	7	10	14

To begin, the process P0 appears at time 0. As a result, the CPU is assigned to P0.

- Process P1 arrived at the time one unit, while process P0 was operating, and the Priority for P0 (4) is less than the priority of process P1 (1). As a result, P1 is allocated to the processor.
- Now, at time 3ms, P1 completes its execution time, and the CPU has to choose a process from the ready queue. While at the ready queue, we have the P2 process arrive at time 3ms.
- Now, among P0 and P2, the CPU will be allocated to the process P2 due to higher priority. And the execution of the process P2 will be started.
- The execution was in progress but at the same time, Process P3 arrived. The priority for P2 (3) is less than P3 (2). So now, P3 started the execution till time equal to 7ms.
- Now P2 again got the chance to complete its execution. So, P2 finishes the execution at time 10ms.
- And finally, at last, process P0 completed its remaining execution till time 14ms.

To calculate Turnaround Time and Waiting time,

Turnaround time = Completion time - Arrival time

Waiting time = Turnaround time - Burst time

Process	Arrival Time	Priority	CPU Burst Time	Completion Time	Turnaround time	Waiting time
P0	0	4	5	14	14	9
P1	1	1	2	3	2	0
P2	3	3	4	10	7	3
P3	4	2	3	7	3	0

Advantages

- Each occurrence resulted in the disruption of ongoing tasks.
- When used in a multiprogramming environment, preemptive scheduling is advantageous.
- The average response time is also improved by using this scheduling strategy.
- The operating system makes sure that all processes use the same amount of CPU.
- Preemptive scheduling is a more reliable approach that prevents one process from dominating the CPU.

Disadvantages

- If many high-priority processes arrive simultaneously, the low-priority process will have to wait longer.
- The scheduler spends more time suspending the ongoing job, switching the context, and dispatching the new incoming task.
- Scheduling requires a limited amount of computational resources.

2 Non Preemptive Priority Scheduling

In the Non Preemptive Priority scheduling, The Processes are scheduled according to the priority number assigned to them. Once the process gets scheduled, it will run till the completion. Generally, the lower the priority number, the higher is the priority of the process.

Example

In the Example, there are 7 processes P1, P2, P3, P4, P5, P6 and P7. Their priorities, Arrival Time and burst time are given in the table.

Process ID	Priority	Arrival Time	Burst Time
1	2	0	3
2	6	2	5
3	3	1	4
4	5	4	2
5	7	6	9
6	4	5	4
7	10	7	10

The Process P1 arrives at time 0 with the burst time of 3 units and the priority number 2. Since No other process has arrived till now hence the OS will schedule it immediately.

Meanwhile the execution of P1, two more Processes P2 and P3 are arrived. Since the priority of P3 is 3 hence the CPU will execute P3 over P2.

Meanwhile the execution of P3, All the processes get available in the ready queue. The Process with the lowest priority number will be given the priority. Since P6 has priority number assigned as 4 hence it will be executed just after P3.

After P6, P4 has the least priority number among the available processes; it will get executed for the whole burst time.

Since all the jobs are available in the ready queue hence All the Jobs will get executed according to their priorities. If two jobs have similar priority number assigned to them, the one with the least arrival time will be executed.

P1	P3	P6	P4	P2	P5	P7	
0	3	7	11	13	18	27	37

From the GANTT Chart prepared, we can determine the completion time of every process. The turnaround time, waiting time and response time will be determined.

Turn Around Time = Completion Time - Arrival Time

Waiting Time = Turn Around Time - Burst Time

Process Id	Priority	Arrival Time	Burst Time	Completion Time	Turnaround Time	Waiting Time	Response Time
1	2	0	3	3	3	0	0
2	6	2	5	18	16	11	13
3	3	1	4	7	6	2	3
4	5	4	2	13	9	7	11
5	7	6	9	27	21	12	18
6	4	5	4	11	6	2	7
7	10	7	10	37	30	18	27

Avg Waiting Time = $(0+11+2+7+12+2+18)/7 = 52/7$ ms

Avg Turn Around Time = $(3+16+6+9+21+6+30)/7 = 91/7 = 13$ ms

Problem with Priority Scheduling Algorithm

In priority scheduling algorithm, the chances of **indefinite blocking** or **starvation**. A process is considered blocked when it is ready to run but has to wait for the CPU as some other process is running currently. But in case of priority scheduling if new higher priority processes keeps coming in the ready queue then the processes waiting in the ready queue with lower priority may have to wait for long durations before getting the CPU for execution.

UNIT 2

PROCESS SYNCHRONIZATION

Process Synchronization is the coordination of execution of multiple processes in a multi-process system to ensure that they access shared resources in a controlled and predictable manner. It aims to resolve the problem of race conditions and other synchronization issues in a concurrent system.

The main objective of process synchronization is to ensure that multiple processes access shared resources without interfering with each other, and to prevent the possibility of inconsistent data due to concurrent access.

To achieve this, various synchronization techniques such as **semaphores, monitors, and critical sections** are used.

In a multi-process system, synchronization is necessary to ensure data consistency and integrity, and to avoid the risk of deadlocks and other synchronization problems.

Process synchronization problem arises in the case of Cooperative process also because resources are shared in Cooperative processes.

Race Condition:

- When more than one process is executing the same code or accessing the same memory or any shared variable in that condition there is a possibility that the output or the value of the shared variable is wrong so for that all the processes doing the race to say that my output is correct this condition known as a race condition.
- Several processes access and process the manipulations over the same data concurrently, then the outcome depends on the particular order in which the access takes place.
- A race condition is a situation that may occur inside a critical section. This happens when the result of multiple thread execution in the critical section differs according to the order in which the threads execute.
- Race conditions in critical sections can be avoided if the critical section is treated as an atomic instruction. Also, proper thread synchronization using locks or atomic variables can prevent race conditions.

Critical Section Problem:

A critical section is a code segment that can be accessed by only one process at a time. The critical section contains shared variables that need to be synchronized to maintain the consistency of data variables. So the critical section problem means designing a way for cooperative processes to access shared resources without creating data inconsistencies.


```
do {  
    flag[i] = TRUE ;  
    turn = j ;  
    while (flag[j] && turn == j) ;  
    critical section  
    flag[i] = FALSE ;  
    remainder section  
} while (TRUE) ;
```

In the entry section, the process requests for entry in the **Critical Section**.

Any solution to the critical section problem must satisfy three requirements:

- **Mutual Exclusion:** If a process is executing in its critical section, then no other process is allowed to execute in the critical section.
- **Progress:** If no process is executing in the critical section and other processes are waiting outside the critical section, then only those processes that are not executing in their remainder section can participate in deciding which will enter in the critical section next, and the selection cannot be postponed indefinitely.
- **Bounded Waiting:** A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

PETERSON'S SOLUTION:

Peterson's Solution is a classical software-based solution to the critical section problem. In Peterson's solution, we have two shared variables:

- boolean flag[i]: Initialized to FALSE, initially no one is interested in entering the critical section
- int turn: The process whose turn is to enter the critical section.

```
do {  
    flag[i] = TRUE ;  
    turn = j ;  
    while (flag[j] && turn == j) ;  
    critical section  
    flag[i] = FALSE ;  
    remainder section  
} while (TRUE) ;
```

Peterson's Solution preserves all three conditions:

- Mutual Exclusion is assured as only one process can access the critical section at any time.
- Progress is also assured, as a process outside the critical section does not block other processes from entering the critical section.
- Bounded Waiting is preserved as every process gets a fair chance.

Disadvantages of Peterson's solution:

- It involves busy waiting. (In the Peterson's solution, the code statement- "while(flag[j] && turn == j);" is responsible for this. Busy waiting is not favored because it wastes CPU cycles that could be used to perform other tasks.)
- It is limited to 2 processes.
- Peterson's solution cannot be used in modern CPU architectures.

BAKERY ALGORITHM

The **Bakery algorithm** is one of the simplest known solutions to the mutual exclusion problem for the general case of N process. Bakery Algorithm is a critical section solution for N processes. The algorithm preserves the first come first serve property.

In the Bakery Algorithm, each process is assigned a number (a ticket) in a lexicographical order. Before entering the critical section, a process receives a ticket number, and the process with the smallest ticket number enters the critical section. If two processes receive the same ticket number, the process with the lower process ID is given priority.

- Before entering its critical section, the process receives a number. Holder of the smallest number enters the critical section.
- If processes P_i and P_j receive the same number,

```
if  $i < j$ 
     $P_i$  is served first;
else
     $P_j$  is served first.
```

- The numbering scheme always generates numbers in increasing order of enumeration; i.e., 1, 2, 3, 3, 3, 3, 4, 5, ...

Pseudo code:

```
repeat
    choosing[i] := true;
    number[i] := max(number[0], number[1], ..., number[n - 1]) + 1;
    choosing[i] := false;
    for j := 0 to n - 1
```

```
do begin
    while choosing[j] do no-op;
    while number[j] != 0
        and (number[j], j) < (number[i], i) do no-op;
end;
critical section

number[i] := 0;

remainder section
until false;
```

Explanation – Firstly the process sets its “choosing” variable to be TRUE indicating its intent to enter critical section. Then it gets assigned the highest ticket number corresponding to other processes. Then the “choosing” variable is set to FALSE indicating that it now has a new ticket number. This is in-fact the most important and confusing part of the algorithm. It is actually a small critical section in itself ! The very purpose of the first three lines is that if a process is modifying its TICKET value then at that time some other process should not be allowed to check its old ticket value which is now obsolete. This is why inside the for loop before checking ticket value we first make sure that all other processes have the “choosing” variable as FALSE. After that we proceed to check the ticket values of processes where process with least ticket number/process id gets inside the critical section. The exit section just resets the ticket value to zero.

Advantages of Bakery Algorithm:

- **Fairness:** The Bakery Algorithm provides fairness, as it ensures that all processes get a fair chance to access the critical section, and no process will be left waiting indefinitely.
- **Easy to Implement:** The algorithm is easy to understand and implement, as it uses simple concepts such as turn numbers and flags to ensure mutual exclusion.
- **No Deadlock:** The Bakery Algorithm ensures that there is no deadlock situation in the system.
- **No starvation:** The algorithm also ensures that there is no starvation of any process, as every process gets a fair chance to enter the critical section.

Disadvantages Bakery Algorithm:

- **Not Scalable:** The Bakery Algorithm is not scalable, as the overhead of the algorithm increases with the number of processes in the system.

- **High Time Complexity:** The algorithm has a high time complexity, which increases as the number of processes in the system increases. This can result in performance issues in systems with a large number of processes.
- **Busy Waiting:** The algorithm requires busy waiting, which can lead to wastage of CPU cycles and increased power consumption.
- **Memory Overhead:** The algorithm requires extra memory to store the turn number and flag values, which can lead to increased memory overhead in systems with a large number of processes.

SEMAPHORES:

- A semaphore is a signaling mechanism and a thread that is waiting on a semaphore can be signaled by another thread. This is different than a mutex as the mutex can be signaled only by the thread that is called the wait function.
- A semaphore uses two atomic operations, wait and signal for process synchronization. A Semaphore is an integer variable, which can be accessed only through two operations wait () and signal ().

There are two types of semaphores: Binary Semaphores and Counting Semaphores.

- **Binary Semaphores:** They can only be either 0 or 1. They are also known as mutex locks, as the locks can provide mutual exclusion. All the processes can share the same mutex semaphore that is initialized to 1. Then, a process has to wait until the lock becomes 0. Then, the process can make the mutex semaphore 1 and start its critical section. When it completes its critical section, it can reset the value of the mutex semaphore to 0 and some other process can enter its critical section.
- **Counting Semaphores:** They can have any value and are not restricted over a certain domain. They can be used to control access to a resource that has a limitation on the number of simultaneous accesses. The semaphore can be initialized to the number of instances of the resource. Whenever a process wants to use that resource, it checks if the number of remaining instances is more than zero, i.e., the process has an instance available. Then, the process can enter its critical section thereby decreasing the value of the counting semaphore by 1. After the process is over with the use of the instance of the resource, it can leave the critical section thereby adding 1 to the number of available instances of the resource.

Advantages of Process Synchronization:

- Ensures data consistency and integrity
- Avoids race conditions
- Prevents inconsistent data due to concurrent access
- Supports efficient and effective use of shared resources

Disadvantages of Process Synchronization:

- Adds overhead to the system.
- Can lead to performance degradation.
- Increases the complexity of the system.
- Can cause deadlocks if not implemented properly.

CLASSICAL PROBLEMS OF SYNCHRONIZATION WITH SEMAPHORE SOLUTION

- In our solutions to the problems, we use semaphores for synchronization, since that is the traditional way to present such solutions. However, actual implementations of these solutions could use mutex locks in place of binary semaphores.
- These problems are used for testing nearly every newly proposed synchronization scheme. The following problems of synchronization are considered as classical problems:
 1. Bounded-buffer (or Producer-Consumer) Problem,
 2. Dining-Philosophers Problem,
 3. Readers and Writers Problem,
 4. Sleeping Barber Problem

READERS AND WRITERS PROBLEM:

Suppose that a database is to be shared among several concurrent processes. Some of these processes may want only to read the database, whereas others may want to update (that is, to read and write) the database. We distinguish between these two types of processes by referring to the former as readers and to the latter as writers. Precisely in OS we call this situation as the readers-writers problem. Problem parameters:

- One set of data is shared among a number of processes.
- Once a writer is ready, it performs its write. Only one writer may write at a time.
- If a process is writing, no other process can read it.
- If at least one reader is reading, no other process can write.
- Readers may not write and only read.

Readers Writer Problem

Problem Statement

- There is a shared resource which should be accessed by multiple processes.
- There are two types of processes in this context. They are reader and writer.
- Any number of readers can read from the shared resource simultaneously, but only one writer can write to the shared resource.
- When a writer is writing data to the resource, no other process can access the resource.

- A writer cannot write to the resource if there are nonzero numbers of readers accessing the resource at that time.

Solution using semaphore:

- From the above problem statement, If a writer wants to write to the resource, it must wait until there are no readers currently accessing that resource.
- Here, we use one mutex = 1 and a semaphore S=1.
- An integer variable read_count is used to maintain the number of readers currently accessing the resource.
- The variable read_count is initialized to 0.

Writer

```
do
{
    wait(S);
    /* perform the write
    operation */
    signal(S);
} while(TRUE)
```

Reader

```
do
{
    //acquire lock
    wait(mutex);
    read_count++;
    if(read_count == 1)
        wait(S);
    //release lock
    signal(mutex);
    /* perform the reading
    operation */
    // acquire lock
    wait(mutex);
    read_count--;
    if(read_count == 0)
        signal(S);
    // release lock
    signal(mutex);
} while(TRUE)
```

Code explanation

- As seen above in the code for the writer, the writer just waits on the w semaphore until it gets a chance to write to the resource.
- After performing the write operation, it increments w so that the next writer can access the resource.
- On the other hand, in the code for the reader, the lock is acquired whenever the read_count is updated by a process.
- When a reader wants to access the resource, first it increments the read_count value, then accesses the resource and then decrements the read_count value.
- The semaphore w is used by the first reader which enters the critical section and the last reader which exits the critical section.

- The reason for this is, when the first readers enters the critical section, the writer is blocked from the resource. Only new readers can access the resource now.
- Similarly, when the last reader exits the critical section, it signals the writer using the w semaphore because there are zero readers now and a writer can have the chance to access the resource.

DEADLOCKS

A **deadlock** is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.

A process requests resources; if the resources are not available the process enters a wait state. Waiting processes may never change its state because the resources they have requested are held by other waiting processes. This situation is called a “deadlock”.

Example 1: System has 2 disk drives

P1 and P2 each hold one disk drive and each needs another one.

Example 2: When two trains approach each other at a crossing, both shall come to a full stop and neither shall start up again until the other has gone away.

To illustrate deadlock state, consider a system with three type drives. Suppose each of three processes holds one of these tape drives. If each processes one request another tape drive, the three processes will be in a deadlock state. Each is waiting for the event “tape drive is released,” which can be caused only by one of the waiting processes. This eg illustrates a deadlock involving the sane resource tape.

Deadlock may also involve different resources type for eg, consider a system with one printer and one tape drive. Suppose that process P1 is holding the tape drive and process P1 request a tape drive, a deadlock occurs.

A process may utilize a resource in only the fowling sequence:

- 1. Request:** If the request can't be granted immediately (for eg, the resource is being used by another process) then the requested process must wait until it acquire the resource.
- 2. Use:** The process can operate on the resource(for eg, if the resources is a printer the process can print on the printer).
- 3. Release:** The process releases the resource.

DEADLOCK CHARACTERIZATION

In a deadlock, processes never finish executing and system resources are tied up, preventing other jobs from starting.

Necessary Conditions

A deadlock situation can arise if the following four conditions hold simultaneously in a system.

1. Mutual Exclusion: At least one resource must be held in a non-sharable mode that is, only one process at a time can use the resources. If another process request that resources, the requesting process must be delayed until the resource has been released.

2. Hold and Wait: A process must be holding at least one resource and waiting to acquire additional resources that are currently being held by the other processes.

3. No preemption: Resources cannot be preempted; that is, a resource can be released voluntarily by the process holding it, after that process has completed its task.

4. Circular Wait: A set $(P_0, P_1, \dots, P_n)$ of waiting processes must exists such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by $P_2, \dots, P_{n-1}$ is waiting for a resource that is held by P_n , and P_n is waiting for a resource that is held by P_0 .

RESOURCE-ALLOCATION GRAPH (RAG)

Deadlock can be described more precisely in terms of a direct graph called a **Resource-allocation graph**.

The Resource Allocation graph mainly consists of a **set of vertices V** and a **set of Edges E**.

1. Vertices: There are two kinds of vertices used in the resource allocation graph

Process Vertices $P = \{P_1, P_2, \dots, P_n\}$, set of all the processes

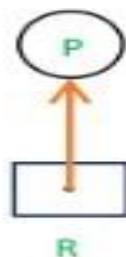
Resource Vertices $R = \{R_1, R_2, \dots, R_m\}$, set of all the resource types

2. Edges: There are two kinds of edges used in the resource allocation graph and these are:

Request edge: Process is requesting a resource – directed edge $P \rightarrow R$



Assignment edge: Resource is assigned to a process – directed edge $R \rightarrow P$



- **Process** is represented as: 
- **Resource** is represented as: 

- **Single instance type resource** – It represents as a box, inside the box, there will be one dot. So the number of dots indicates how many instances are present of each resource type.

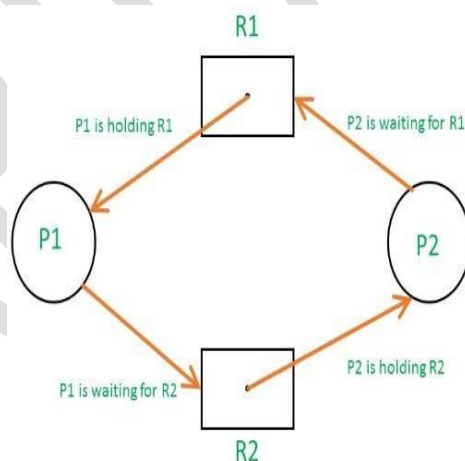


- **Multi-resource instance type resource** – It also represents as a box, inside the box, there will be many dots present.



1 . Single instance resource allocation graph

If there is a cycle in the Resource Allocation Graph and each resource in the cycle provides only one instance, then the processes will be in deadlock. For example, if process P1 holds resource R1, process P2 holds resource R2 and process P1 is waiting for R2 and process P2 is waiting for R1, then process P1 and process P2 will be in deadlock.

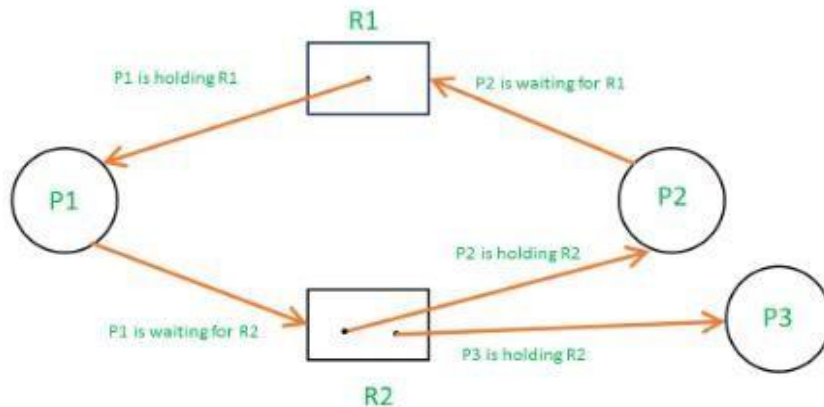


Single instance of resource type

(Note: No Cycles → No Deadlock. If there is a cycle Resource type has exactly one instance → Deadlock . Resource type has several instances → may or may not be a Deadlock)

2 Multiple instance of resource type

It means that resource has multiple instances.



Multiple instances but no deadlock

METHODS FOR HANDLING DEADLOCK

Principally, we can deal with the deadlock problem in one of the of the three ways:

- We can use a protocol to prevent or avoid deadlock, ensuring that the system will never enter a deadlock state.
- We can allow system to enter a deadlock state, detect it, and recover.
- We can ignore the problem altogether, and pretend that deadlock never occur in the system. This solution is used by most operating systems, including UNIX.

DEADLOCK PREVENTION

For a deadlock to occur each of the four necessary conditions must hold. By ensuring that at least one of these conditions cannot hold, we can prevent the occurrence of a deadlock.

1. Mutual Exclusion
2. Hold And Wait
3. No Preemption
4. Circular Wait

1. Mutual Exclusion

This condition must hold for non-sharable resources. For example, a printer cannot be simultaneously shared by several processes. In contrast, Sharable resources do not require mutually exclusive access and thus cannot be involved in a deadlock. A good example of a sharable resource is Read-only files because if several processes attempt to open a read-only file at the same time, then they can be granted simultaneous access to the file.

A process need not to wait for the sharable resource. Generally, deadlocks cannot be prevented by denying the mutual exclusion condition because there are some resources that are intrinsically non-sharable.

2. Hold And Wait

Hold and wait condition occurs when a process holds a resource and is also waiting for some other resource in order to complete its execution. Thus if we did not want the occurrence of this condition then we must guarantee that when a process requests a resource, it does not hold any other resource.

There are some protocols that can be used in order to ensure that the Hold and Wait condition never occurs:

- According to the first protocol; each process must request and gets all its resources before the beginning of its execution.
- The second protocol allows a process to request resources only when it does not occupy any resource.

Let us illustrate the difference between these two protocols:

Consider a process that mainly copies data from a DVD drive to a file on disk, sorts the file, and then prints the results to a printer. If all the resources must be requested at the beginning of the process according to the first protocol, then the process requests the DVD drive, disk file, and printer initially. It will hold the printer during its entire execution, even though the printer is needed only at the end.

While the second method allows the process to request initially only the DVD drive and disk file. It copies the data from the DVD drive to the disk and then releases both the DVD drive and the disk file. The process must then again request the disk file and printer. After copying the disk file to the printer, the process releases these two resources as well and then terminates.

Disadvantages of Both Protocols

- Utilization of resources may be low, since resources may be allocated but unused for a long period. In the above-given example, for instance, we can release the DVD drive and disk file and again request the disk file and printer only if we can be sure that our data will remain on the disk file. Otherwise, we must request all the resources at the beginning of both protocols.
- There is a possibility of starvation. A process that needs several popular resources may have to wait indefinitely because at least one of the resources that it needs is always allocated to some other process.

3. No Preemption

The third necessary condition for deadlocks is that there should be no preemption of resources that have already been allocated. In order to ensure that this condition does not hold the following **protocols can be used:**

- According to the first Protocol: "If a process that is already holding some resources requests another resource and if the requested resources cannot be allocated to it, then it must release all the resources currently allocated to it."
- According to the Second Protocol: "When a process requests some resources, if they are available, then allocate them. If in case the requested resource is not available then we will check whether it is being

used or is allocated to some other process waiting for other resources. If that resource is not being used, then the operating system preempts it from the waiting process and allocates it to the requesting process. And if that resource is being used, then the requesting process must wait".

The second protocol can be applied to those resources whose state can be easily saved and restored later for example CPU registers and memory space, and cannot be applied to resources like printers and tape drivers.

4. Circular Wait

The Fourth necessary condition to cause deadlock is circular wait, In order to violate this condition we can do the following:

Assign a priority number to each resource. There will be a condition that any process cannot request for a lesser priority resource. This method ensures that not a single process can request a resource that is being utilized by any other process and due to which no cycle will be formed.

Example: Assume that R5 resource is allocated to P1, if next time P1 asks for R4, R3 that are lesser than R5; then such request will not be granted. Only the request for resources that are more than R5 will be granted.

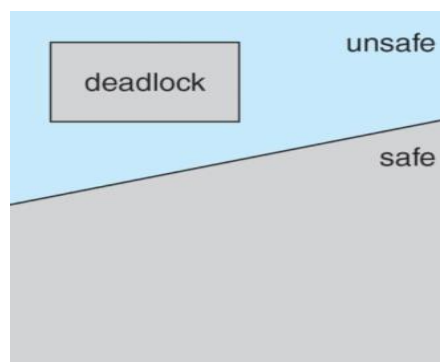
DEADLOCK AVOIDANCE

An alternative method for avoiding deadlock is to require additional information about how resources are to be requested. Each request requires that the system consider the resources currently available, the resources currently allocated to each process, & the future request and releases of each process, to decide whether the current request can be satisfied or must wait to avoid a possible future deadlock.

Safe State - If the system can allocate resources to the process in such a way that it can avoid deadlock. Then the system is in a safe state.

Unsafe State- If the system can't allocate resources to the process safely, then the system is in an unsafe state.

Note: *Unsafe state not always cause deadlock.*



Safe, Unsafe and deadlock state spaces

A safe state is not a deadlock state. Conversely, a deadlock is an unsafe state. Not all unsafe state are deadlock, however, a unsafe may lead to a deadlock. As long as the state is safe, the operating system can avoid unsafe state.

Example: To illustrate, we consider a system with 12 magnetic tape drives and three processes:P0, P1 and P2. Process P0 requires 10 tape drivers, process P1 may need as many as 4, and process P2 may need up to 9 tape drives.

Suppose that, at time **t0**, process P0 is holding 5 tape drivers, process **P1** is holding 2, and process **P2** is holding 2 tape drives.(Thus, there are 3 free tape drives)

	Maximum Needs	Current Needs
P0	10	5
P1	4	2
P2	9	2

At time **t0**, a system is in safe state. The sequence<**P1, P0, P2**> satisfies the safety condition, since process **P1** can immediately be allocated all its tape drives and then return them (the system will then have 5 available tape drives),

Then process **P0** can get all its tape drives and return them. The system will then have 10 available tape drives, and finally process **P2** could get all its tape drives return them (the system will then have all 12 tape drives available).

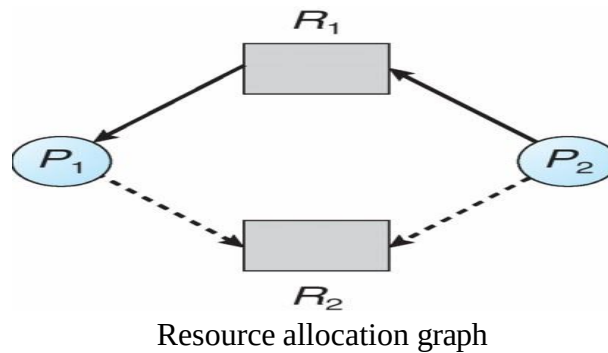
A system goes from a safe state to an unsafe state. Suppose that, at time **t1**, process **P2** request and is allocated one more tape drive. The system is no longer in a safe state. At this point, only process **P1** can be allocated all its tape drives. When it returns them, the system will have only 4 available tape drives. Since process **P0** is allocated 5 tapes drives, but has a maximum of 10, it may then request 5 more tape drives since they are unavailable process **P0** must wait. Similarly, process **P2** may request an additional 6 tape drives and have to wait, resulting in a deadlock.

The following are the avoidance algorithms

1. Resource-Allocation Graph Algorithm

A Claim edge **Pi->Rj** indicated that process **Pi** may request resource **Rj**; represented by a dashed line. A Claim edge converts to request edge when a process requests a resource. A Request edge converted to an assignment edge when the resource is allocated to the process.

When a resource is released by a process, assignment edge reconverts to a claim edge. Resources must be claimed a prior in the system



Suppose that process P_i requests a resource R_j .

The request can be granted only if converting the request edge to an assignment edge does not result in the formation of a cycle in the resource allocation graph

2. Bankers Algorithm

The resource allocation graph algorithm is not applicable to resource allocation system with multiple instance of each resource type. So as a solution this algorithm was introduced and is commonly known as BANKERS ALGORITHM. The name was chosen because this algorithm could be used in the banking system to ensure that bank never allocates available cash such that it can no longer satisfy the needs of all its customers.

When a new process enters the system, it must declare the maximum number of instance of each resource type that may need. This number may not exceed the total number of resources in the system. When a user request a set of resources, the system must determine whether the allocation of these resources will leave the system in a safe state. If it will, the resources are allocated; otherwise the process must wait until some other process releases enough resources.

Data Structure of Bankers Algorithm:

- **Available:** A vector of length m indicates the number of available resources of each type. If $\text{Available}[j] = k$, there are k instances of resource type R_j available.
- **Max:** An $n \times m$ matrix defines the maximum demand of each process. If $\text{max}[i, j] = k$, then process P_i may request at most k instances of resource type R_j .
- **Allocation:** An $n \times m$ matrix defines the number of resource of each type currently allocated to each process. If $\text{Allocation}[i, j] = k$, then process P_i is currently allocated k instances of resource type R_j .
- **Need:** An $n \times m$ matrix indicates the remaining resources need of each process. If $\text{Need}[i, j] = k$, then process P_i may need k more instance of resource type R_j , to complete its task. Note that $\text{Need}[i, j] = \text{max}[i, j] - \text{Allocation}[i, j]$.

Example: There is a process table with number of processes that contains **allocation field** (for showing the number of resources of type: A, B and C allocated to each process in the table), **max field** (for showing the maximum number of resources of type: A, B, and C that can be allocated to each process) and also, the **available field** (for showing the currently available resources of each type in the table).

Processes	Allocation	Max	Available
	A B C	A B C	A B C
P0	1 1 2	5 4 4	3 2 1
P1	2 1 2	4 3 3	
P2	3 0 1	9 1 3	
P3	0 2 0	8 6 4	
P4	1 1 2	2 2 3	

Q.1 Calculate the need matrix? Q.2 is the system in a safe state?

Ans.1 Calculate the entries of need matrix using the formula: **(Need)_i = (Max)_i – (Allocation)_i**

Process	Need
	A B C
P0	4 3 2
P1	2 2 1
P2	6 1 2
P3	8 4 4
P4	1 1 1

Ans.2 Let us check for safe sequence:

1. For process P0, Need = (4, 3, 2) and Available = (3, 2, 1)

Clearly, the resources needed are more in number than the available ones. So, now the system will move to process the next request.

2. For Process P1, Need = (2, 2, 1) and Available = (3, 2, 1)

Clearly, resources needed are less than equal to the available resources within the system. Hence, request of P1 is granted.

Available = Available + Allocation = (3, 2, 1) + (2, 1, 2) = (5, 3, 3) (New Available)

3. For Process P2, Need = (6, 1, 2) and Available = (5, 3, 3)

The resources needed are more in number than the available ones. So, now the system will move to process the next request.

4. For Process P3, Need = (8, 4, 4) and Available = (5, 3, 3)

The resources needed are more in number than the available ones. So, now the system will move to process the next request.

5. For Process P4, Need = (1, 1, 1) and Available = (5, 3, 3)

Resources needed are less than equal to the available resources within the system. Hence, request of P4 is granted.

Available=Available + Allocation = (5, 3, 3) + (1, 1, 2) = (6, 4, 5) (New Available)

6. Now again check for Process P2, Need = (6, 1, 2) and Available = (6, 4, 5)

Resources needed are less than equal to the available resources within the system. Hence, request of P2 is granted.

Available = Available + Allocation = (6,4,5)+(3,0,1)=(9,4,6) (New Available)

7. Now again check for Process P3, Need = (8, 4, 4) and Available = (9, 4, 6)

Resources needed are less than equal to the available resources within the system. Hence, request of P3 is granted.

Available=Available+Allocation = (9,4,6)+(0,2,0)=(9,6,6)(NewAvailable)

8. Check for Process P0, Need = (4, 3, 2), and Available (9, 6, 6)

Clearly, the request for P0 is also granted. **Safe sequence:** <P1,P4,P2,P3,P0>

The system has allocated all the required number of resources to each process in a particular sequence. Therefore, it is proved that the system is in a safe state.

3. SAFETY ALGORITHM

The algorithm for finding out whether or not a system is in a safe state can be described as follows:

1) Let **Work** and **Finish** be vectors of length '**m**' and '**n**' respectively.

Initialize: **Work = Available**

Finish[i] = false; for i=1, 2, 3, 4....n

2) Find an **i** such that both

a) **Finish[i] = false**

b) **Need i <= Work**

if no such **i** exists go to step (4)

3) **Work = Work + Allocation[i]**

Finish[i] = true

go to step (2)

4) If **Finish [i] = true** for all **i** then the system is in a safe state. This algorithm may require an order of **m x n²** operations to decide whether a state is safe.

4. RESOURCE-REQUEST ALGORITHM

Let Request_i be the request vector for process P_i . If $\text{Request}_i[j] = k$, then process P_i wants k instances of resource type R_j . When a request for resources is made by process P_i , the following actions are taken:

Let Request_i be the request array for process P_i . $\text{Request}_i[j] = k$ means process P_i wants k instances of resource type R_j . When a request for resources is made by process P_i the following actions are taken:

1) If $\text{Request}_i \leq \text{Need}_i$ go to step (2)

Otherwise, raise an error condition, since the process has exceeded its maximum claim.

2) If $\text{Request}_i \leq \text{Available}$

go to step (3); otherwise, P_i must wait, since the resources are not available.

3) Have the system pretend to have allocated the requested resources to process P_i by modifying the state as follows:

$$\text{Available} = \text{Available} - \text{Request}_i$$

$$\text{Allocation}_i = \text{Allocation}_i + \text{Request}_i$$

$$\text{Need}_i = \text{Need}_i - \text{Request}_i$$

If the resulting resource-allocation state is safe, the transaction is completed and process P_i is allocated its resources. However, if the new state is unsafe, then P_i must wait for Request_i and the old resource-allocation state is restored.

DEADLOCK DETECTION

If a system does not employ either a deadlock-prevention or a deadlock avoidance algorithm, then a deadlock situation may occur. In this environment, the system must provide:

- An algorithm that examines the state of the system to determine whether a deadlock has occurred
- An algorithm to recover from the deadlock

Deadlock Detection

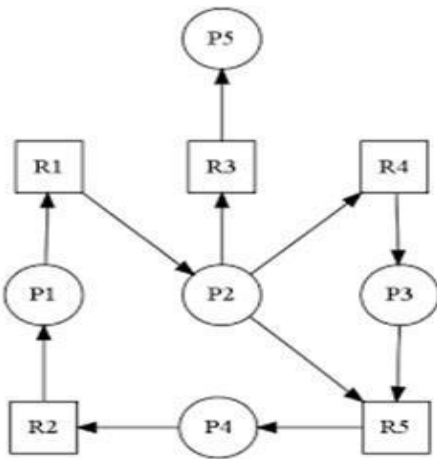
There are two deadlock detection methods depending upon the number of instances of each resource: 1. Single instance of each resource 2. Multiple instance of each resource

Single Instance of Each Resource: *wait-for-graph*

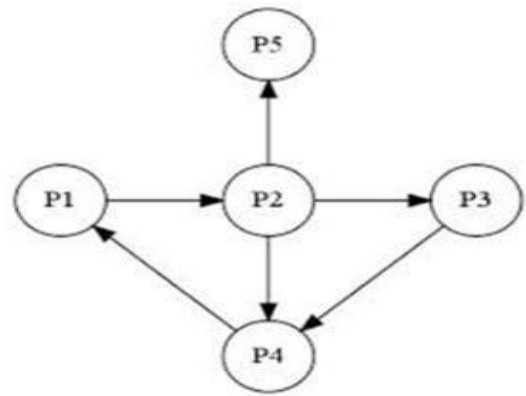
When there is a single instance of each resource the system can use wait-for-graph for deadlock detection.

The key points about the wait-for-graph are:

1. Obtained from resource allocation graph
2. Remove the resource nodes – from the resource allocation graph remove the resource nodes
3. Collapse the corresponding edges



(a) Resource allocation graph



(b) Corresponding Wait-for graph

An Edge from **P_i** to **P_j** means that process **P_i** is waiting for a resource held by process **P_j**. Now, if there are two edges **P_i R_q** and **R_q P_j** in resource allocation graph, for some resource **R_q** the collapse these into one single edge from **P_i P_j** to make the wait-for-graph. Finally, if there is a cycle in wait-for-graph, then the system is in deadlock else not. For example, in the above figure, **P₂** is requesting **R₃** which is held by **P₅**. Hence, we remove the resource **R₃** and collapse the edge between **P₂** and **P₅**. Wait-for-graph reflects this change. Similarly, the collapse of edges between **P₄** and **P₁** and all other processes is done

Several Instances of a Resource Type

The wait-for graph scheme is not applicable to a resource-allocation system with multiple instances of each resource type.

Data Structure

- **Available:** A vector of length m indicates the number of available resources of each type.
- **Allocation:** An $n \times m$ matrix defines the number of resources of each type currently allocated to each process.
- **Request:** An $n \times m$ matrix indicates the current request of each process.

If **Request [i, j] = k**, then process **P_i** is requesting k more instances of resource type **R_j**.

The $\leq$ relation between two vectors is defined as in bankers algorithm. To simplify notation, we shall again treat the rows in the matrices Allocation and Request as vectors, and shall refer to them as **Allocation_i** and **Request_i**, respectively.

1. Let **Work** and **Finish** be vectors of length m and n , respectively.

Initialize, **Work** := **Available**. For $i = 1, 2, \dots, n$,

If **Allocation_i** != 0, then **Finish [i]** := false; otherwise, **Finish[i]** := true.

2. Find an **index i** such that both

a. **Finish[i] = false**

b. **Request_i ≤ Work**

If no such exists, go to step 4

3. **Work**: = **Work** + **Allocation_i**

Finish[i] := true

go to step 2.

4. If **Finish[i] = false**, for some **i**, $1 \leq i \leq n$, then the system is in a deadlock state. Moreover, if **Finish[i] = false**, then process **P_i** is deadlocked.

This algorithm requires an order of $m \times n^2$ operations to detect whether the system is in a deadlocked state.

Example: To illustrate this algorithm, we consider a system with five processes P0 through P4 and three resource types A, B, C. Resource type A has 7 instances, resource type B has 2 instances, and resource type C has 6 instances. Suppose that, at time To, we have the following resource-allocation state:

	Allocation	Request	Available
	ABC	ABC	ABC
P0	010	000	000
P1	200	202	
P2	303	000	
P3	211	100	
P4	002	002	

We claim that the system is not in a deadlocked state. Indeed, if we execute our algorithm, we will find that the sequence **<P0, P2, P3, P1, P4>** will result in **Finish[i] = true** for all **i**.

Suppose now that process P2 makes one additional request for an instance of type C. The Request matrix is modified as follows:

	Request
	A B C
P0	0 0 0
P1	2 0 2
P2	0 0 1
P3	1 0 0
P4	0 0 2

Is the system in deadlocked state? Now, Safe sequence is **<P0, P2, P3, P1, P4>** **P0** request can be satisfied with the Available resources. Hence, the first process in safe sequence is P0. Next, update the Available using: **Available=Available + Allocation of P0** So, the new Available is **<0,1,0>** The next process whose Request can be satisfied with updated Available is **P2**. So, the updated safe frequencies is **<P0,P2>** **Available=Available + Allocation** of P2 So, the new Available is **<3,1,3>** Next, in the same manner find a process whose request can be satisfied. So, the final sequence is **<P0, P2, P3, P1, P4>**

Note: There can be multiple safe-sequences. As long as there is one safe-sequence, the system is not in deadlock state.

We claim that the system is now deadlocked. Although we can reclaim the resources held by process P0, the number of available resources is not sufficient to fulfill the requests of the other processes. Thus, a deadlock exists, consisting of processes P1, P2, P3, and P4.

Detection-Algorithm Usage

When should we invoke the detection algorithm? The answer depends on two factors:

1. How often is a deadlock likely to occur?
2. How many processes will be affected by deadlock when it happens?

If deadlocks occur frequently, then the detection algorithm should be invoked frequently. Resources allocated to deadlocked processes will be idle until the deadlock can be broken. In addition, the number of processes involved in the deadlock cycle may grow.

Deadlocks occur only when some process makes a request that cannot be granted immediately. This request may be the final request that completes a chain of waiting processes. In the extreme, we could invoke the deadlock- detection algorithm every time a request for allocation cannot be granted immediately. In this case, we can identify not only the set of processes that is deadlocked, but also the specific process that "caused" the deadlock. (In reality, each of the deadlocked processes is a link in the cycle in the resource graph, so all of them, jointly, caused the deadlock.) If there are many different resource types, one request

may cause many cycles in the resource graph, each cycle completed by the most recent request and "caused" by the one identifiable process.

Of course, invoking the deadlock-detection algorithm for every request may incur a considerable overhead in computation time. A less expensive alternative is simply to invoke the algorithm at less frequent intervals—for example, once per hour, or whenever CPU utilization drops below 40 percent. (A deadlock eventually cripples system throughput and will cause CPU utilization to drop) If the detection algorithm is invoked at arbitrary points in time may be many cycles in the resource graph. We would generally not be able to tell which of the many deadlocked processes "caused" the deadlock.

RECOVERY FROM DEADLOCK

When a Deadlock Detection Algorithm determines that a deadlock has occurred in the system, the system must recover from that deadlock. There are two approaches of breaking a Deadlock:

Process Termination: To eliminate the deadlock, we can simply kill one or more processes. For this, we use two methods:

a) Abort all the Deadlocked Processes Aborting all the processes will certainly break the deadlock, but with a great expense. The deadlocked processes may have computed for a long time and the result of those partial computations must be discarded and there is a probability to recalculate them later.

b) Abort one process at a time until deadlock is eliminated Abort one deadlocked process at a time, until deadlock cycle is eliminated from the system. Due to this method, there may be considerable overhead, because after aborting each process, we have to run deadlock detection algorithm to check whether any processes are still deadlocked. Aborting a process may not be easy. If the process was in the midst of updating a file, terminating it will leave that file in an incorrect state. Similarly if the process was in the midst of printing data on the printer, the system must reset the printer to a correct state before printing the next job.

If the partial termination method is used, then, given a set of deadlocked processes, we must determine which process (or processes) should be terminated in an attempt to break the deadlock. This determination is a policy decision, similar to CPU-scheduling problems. The question is basically an economic one; we should abort those processes the termination of which will incur the minimum cost. Unfortunately, the term minimum cost is not a precise one. Many factors may determine which process is chosen, including:

- What the priority of the process is
- How long the process has computed, and how much longer the process will compute before completing its designated task.
- How and many and what type of resources the process has used (for example whether the resources are simple to preempt)
- How many more resources the process needs in order to complete

- How many processes will need to be terminated
- Whether the process is interactive or batch

2. Resource Preemption: To eliminate deadlocks using resource preemption, we preempt some resources from processes and give those resources to other processes. This method will raise three issues –

(a) Selecting a victim: We must determine which resources and which processes are to be preempted and also the order to minimize the cost.

(b) Rollback: We must determine what should be done with the process from which resources are preempted. One simple idea is total rollback. That means abort the process and restart it.

(c) Starvation: In a system, it may happen that same process is always picked as a victim. As a result, that process will never complete its designated task. This situation is called **Starvation** and must be avoided. One solution is that a process must be picked as a victim only a finite number of times.

UNIT 3

MEMORY MANAGEMENT

- Memory management is the act of managing computer memory
- This provides way to allocate portions of memory to programs on their request and freeing it for reuse when no longer needed
- The management of main memory is critical to the computer system.

FUNCTIONS OF MEMORY MANAGEMENT

- Memory is large array of words or bytes each with its own address
- Keeping track of each memory location whether it is free or allocated.
- Enforcing policies to determine to which processes the specified memory area is to be allocated.
- Allocation of memory to a process as per the policy.
- Reclaim the memory from the process when it finishes its task.
- Program must be brought (from disk) into memory and placed within a process for it to be run
- CPU fetches instructions from memory according to the value of the program counter
- Instructions may cause additional loading from and storing to specific memory addresses
- The concept of CPU scheduling allows a set of processes to share the CPU which increases CPU utilization
- This set of process reside in memory
- The memory is then shared and the resources required to be managed

CONCEPTS OF MEMORY MANAGEMENT

1. Binding of Instructions and Data to Memory

Address binding of instructions and data to physical memory addresses can happen at three different stages.

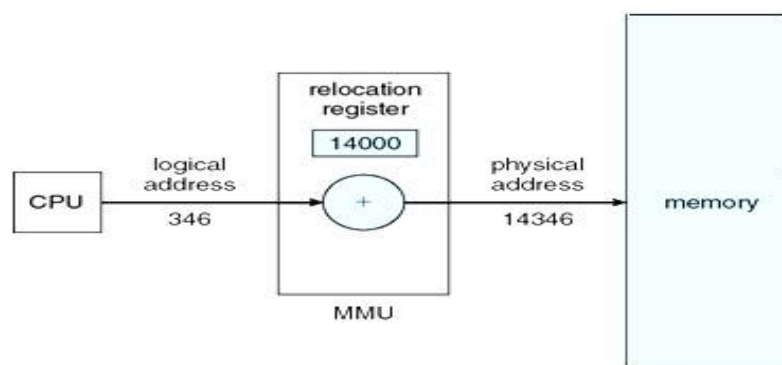
- **Compile time:** While compiling the program itself the physical memory addresses are bound to the instruction and data, **absolute code** can be generated; must recompile code if starting location changes
- **Load time:** Here compiler generates **reloadable code**. The physical addresses are assigned to the instructions and data, while loading the program to the main memory.
- **Execution time:** Binding delayed until run time if the process can be moved during its execution from one memory segment to another. Need hardware to support for address mapping (e.g., base and limit registers.)

2. Logical versus Physical Address

- **Logical Address** A logical address is an address that is generated by the CPU during program execution. The logical address is a virtual address as it does not exist physically, and therefore, it is also known as a **Virtual Address**.
- **Physical Address**- Physical Address is the actual address of the data inside the memory. The logical address is a virtual address and the program needs physical memory for its execution. The user never deals with the Physical Address.
- **Physical Address Space**-Physical address space in a system can be defined as the size of the main memory. It is really important to compare the process size with the physical address space. The process size must be less than the physical address space.
- **Logical Address Space**-Logical address space can be defined as the size of the process. The size of the process should be less enough so that it can reside in the main memory.

Memory-Management Unit (MMU)

- It is a hardware device that maps virtual to physical address
- In MMU scheme, the value in the relocation register is added to every address generated by a user process at the time it is sent to memory
- The user program deals with logical addresses; it never sees the real physical addresses



Dynamic relocation using relocation register

Loading

- **Static Loading:** Static Loading is basically loading the entire program into a fixed address. It requires more memory space.
- **Dynamic Loading:** All the programs are loaded in the main memory for execution. Sometimes complete program is loaded into the memory, but sometimes a certain part or routine of the program is loaded into the main memory only when it is called by the program, this mechanism is called **Dynamic Loading**, this enhance the performance.

- Routine is not loaded until it is called
- Better memory-space utilization; unused routine is never loaded.
- Useful when large amounts of code are needed to handle infrequently occurring cases.
- No special support from the operating system is required; implemented through program design.

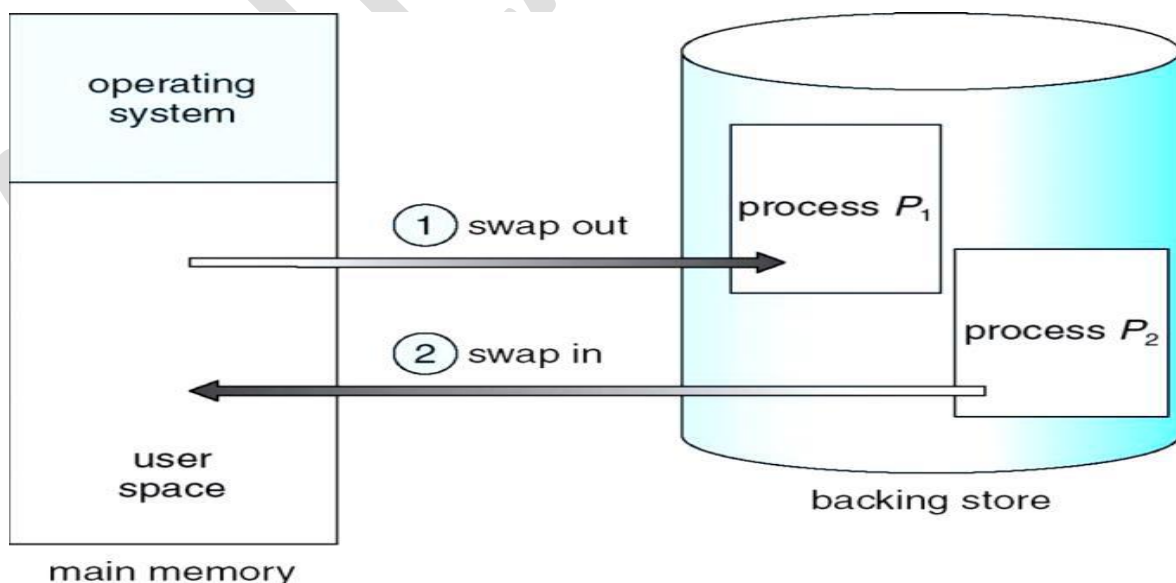
Linking

- **Dynamic Linking:** At times one program is dependent on some other program. In such a case, rather than loading all the dependent programs, CPU links the dependent programs to the main executing program when it is required. This mechanism is known as **Dynamic Linking**.
- **Static Linking:** In static linking, the linker combines all necessary program modules into a single executable program. So there is no runtime dependency. Some operating systems support only static linking, in which system language libraries are treated like any other object module.

Swapping

Swapping is a memory management scheme in which any process can be temporarily swapped from main memory to secondary memory so that the main memory can be made available for other processes. It is used to improve main memory utilization. The purpose of the swapping in operating system is to access the data present in the hard disk and bring it to RAM so that the application programs can use it. It is important to remember that swapping is used only when data is not present in RAM.

Although the process of swapping affects the performance of the system, it helps to run larger and more than one process. This is the reason why swapping is also referred to as memory compaction.



Swapping

Swapping has been subdivided into two concepts: swap-in and swap-out.

- Swap-out is a technique for moving a process from RAM to the hard disc.
- Swap-in is a method of transferring a program from a hard disc to main memory, or RAM.

Advantages

1. It helps the CPU to manage multiple processes within a single main memory.
2. Swapping allows the CPU to perform multiple tasks simultaneously. Therefore, processes do not have to wait very long before they are executed.
3. It improves the main memory utilization.

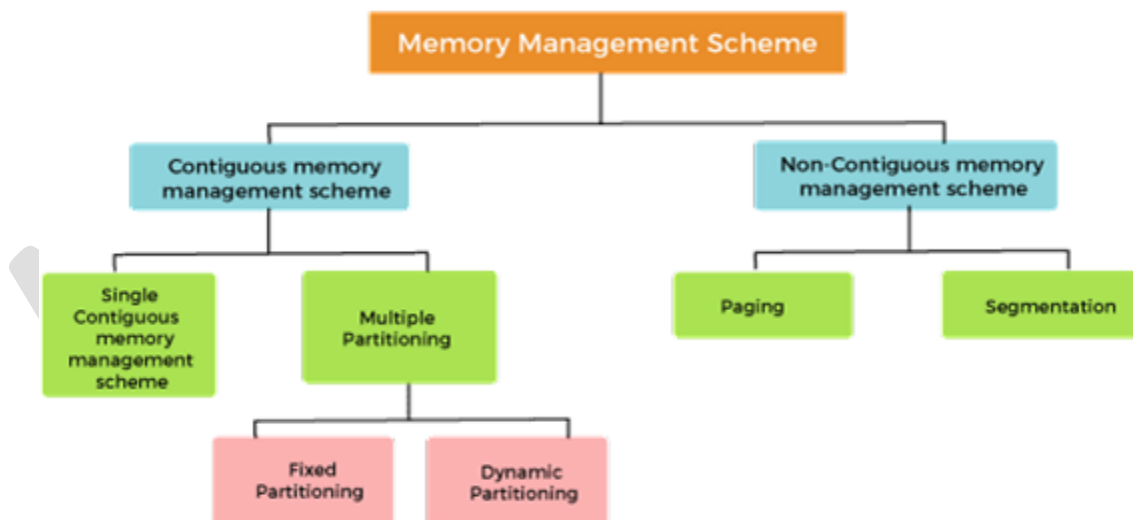
Disadvantages

1. If the computer system loses power, the user may lose all information related to the program in case of substantial swapping activity.
2. If the swapping algorithm is not good, the composite method can increase the number of Page Fault and decrease the overall processing performance.

Memory Management Techniques

The memory management techniques can be classified into following main categories:

1. Contiguous memory management schemes
2. Non-Contiguous memory management schemes



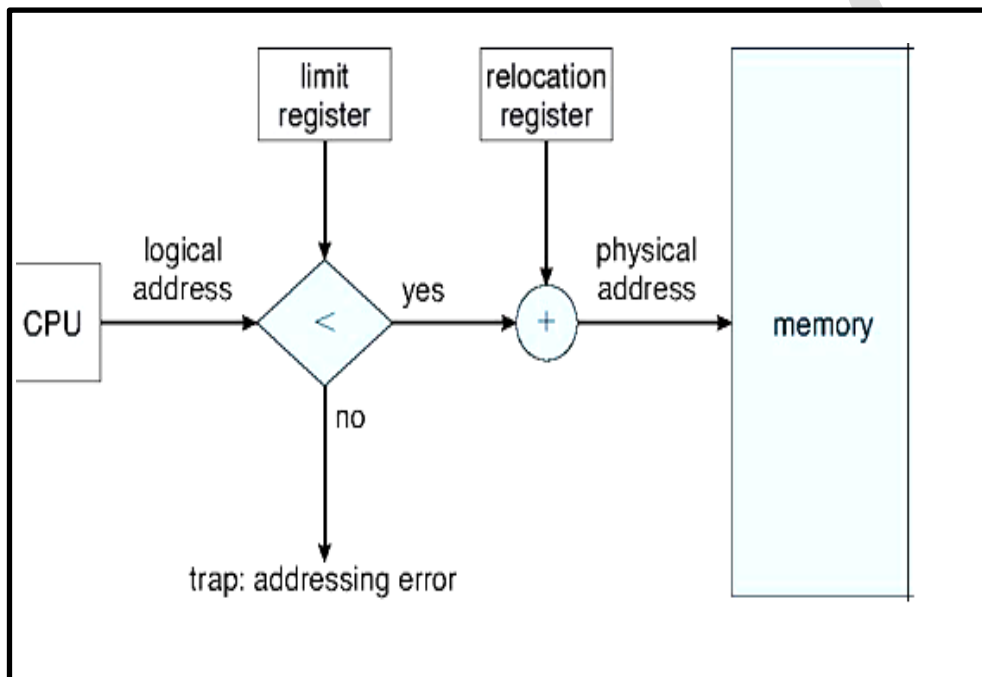
15.5 Classification of memory management schemes

Contiguous Allocation

In a Contiguous memory management scheme, each program occupies a single contiguous block of storage locations, i.e., a set of memory locations with consecutive addresses.

1. Single contiguous memory management

The Single contiguous memory management scheme is the simplest memory management scheme used in the earliest generation of computer systems. In this scheme, the main memory is divided into two contiguous areas or partitions. The operating systems reside permanently in one partition, generally at the lower memory, and the user process is loaded into the other partition. Relocation register is used to protect user processes from each other, and from changing operating-system code and data. Base register contains value of smallest physical address and limit register contains range of logical addresses – each logical address must be less than the limit register. MMU maps logical address *dynamically*.



Hardware Support for Relocation and Limit Registers

Advantages

- Simple to implement.
- Easy to manage and design.
- In a Single contiguous memory management scheme, once a process is loaded, it is given full processor's time, and no other processor will interrupt it.

Disadvantages

- Wastage of memory space due to unused memory as the process is unlikely to use all the available memory space.
- The CPU remains idle, waiting for the disk to load the binary image into the main memory.
- It cannot be executed if the program is too large to fit the entire available main memory space.
- It does not support multiprogramming

2. Partitioned memory

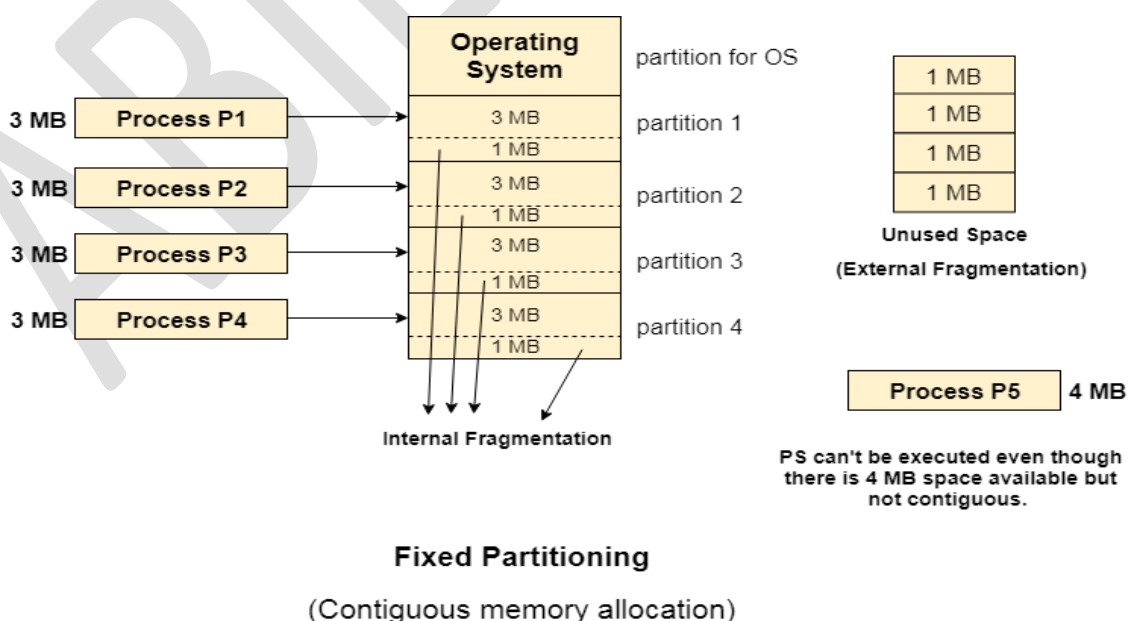
Multiple-partition allocation

In multiprogramming more than one program is to be kept at the same time. To facilitate this, memory is divided into a number of regions or partition. This technique allows multiple jobs reside in the memory simultaneously. Each region can have one program or process. When a region becomes free a program is selected from the job pool and loaded onto that region. When it terminates the region becomes available for another program. Although no special hardware is required to implement this technique it is very much necessary to protect one user memory area by another. Two schemes are possible

1. Static Partition memory management / MFT
2. Dynamic Partitioned Memory Management/MVT

1. Static Partition memory management (MFT- multiple partitioning with fixed size task)

The main memory is partitioned into fixed number of partition with fixed size. The number of partitions to be formed in the memory may be decided by the operator or by operating system itself. Each partition may contain exactly one process. When a partition is free, a process is selected and is loaded into the free Partition. When the process terminates, the partition becomes available for another process. They are not allocated to change as the system runs. Status of each partition and its attributes such as partition number, location and size are maintained in a table called **static partition status table**. Each user job must specify the maximum amount of memory required then a partition equal to or greater than the job size is searched in the static partition table and assigned to the job. This type of static partitioning is a better choice when sizes and frequency of jobs are known well in advance. In this case large amount of memory is wasted in the partition, which cannot be allocated to any other job.



Disadvantages:

1. Internal Fragmentation

If the size of the process is lesser than the total size of the partition then some size of the partition get wasted and remain unused. This is wastage of the memory and called internal fragmentation. As shown in the image below, the 4 MB partition is used to load only 3 MB process and the remaining 1 MB got wasted.

2. External Fragmentation

The total unused space of various partitions cannot be used to load the processes even though there is space available but not in the contiguous form. As shown in the image below, the remaining 1 MB space of each partition cannot be used as a unit to store a 4 MB process. Despite of the fact that the sufficient space is available to load the process, process will not be loaded.

3. Limitation on the size of the process

If the process size is larger than the size of maximum sized partition then that process cannot be loaded into the memory. Therefore, a limitation can be imposed on the process size that is it cannot be larger than the size of the largest partition.

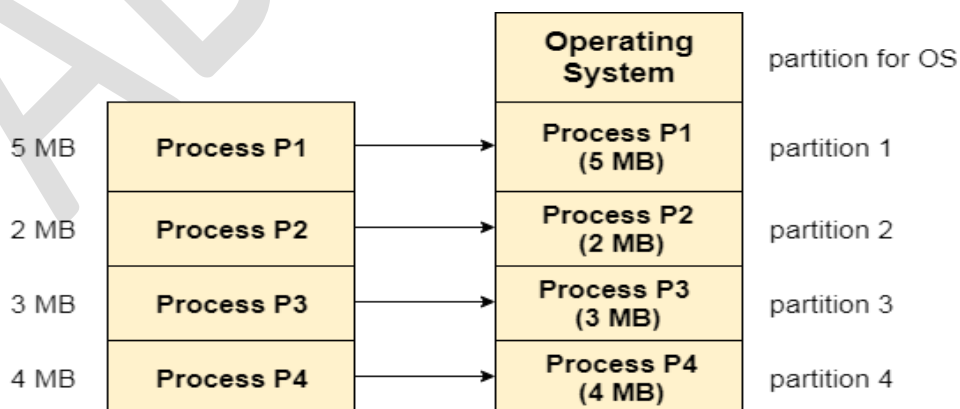
4. Degree of multiprogramming is less

By Degree of multi programming, we simply mean the maximum number of processes that can be loaded into the memory at the same time. In fixed partitioning, the degree of multiprogramming is fixed and very less due to the fact that the size of the partition cannot be varied according to the size of processes.

2. Dynamic Partitioning (MVT-Multiple partitioning with variable size task)

Dynamic partitioning tries to overcome the problems caused by fixed partitioning. In this technique, the partition size is not declared initially. It is declared at the time of process loading.

The first partition is reserved for the operating system. The remaining space is divided into parts. The size of each partition will be equal to the size of the process. The partition size varies according to the need of the process so that the internal fragmentation can be avoided.



Dynamic Partitioning
(Process Size = Partition Size)

Advantages

1. No Internal Fragmentation

In dynamic partitioning partitions are created according to the need of the process, It is clear that there will not be any internal fragmentation because there will not be any unused remaining space in the partition.

2. No Limitation on the size of the process

In Fixed partitioning, the process with the size greater than the size of the largest partition could not be executed due to the lack of sufficient contiguous memory. Here, In Dynamic partitioning, the process size can't be restricted since the partition size is decided according to the process size.

3. Degree of multiprogramming is dynamic

Due to the absence of internal fragmentation, there will not be any unused space in the partition hence more processes can be loaded in the memory at the same time.

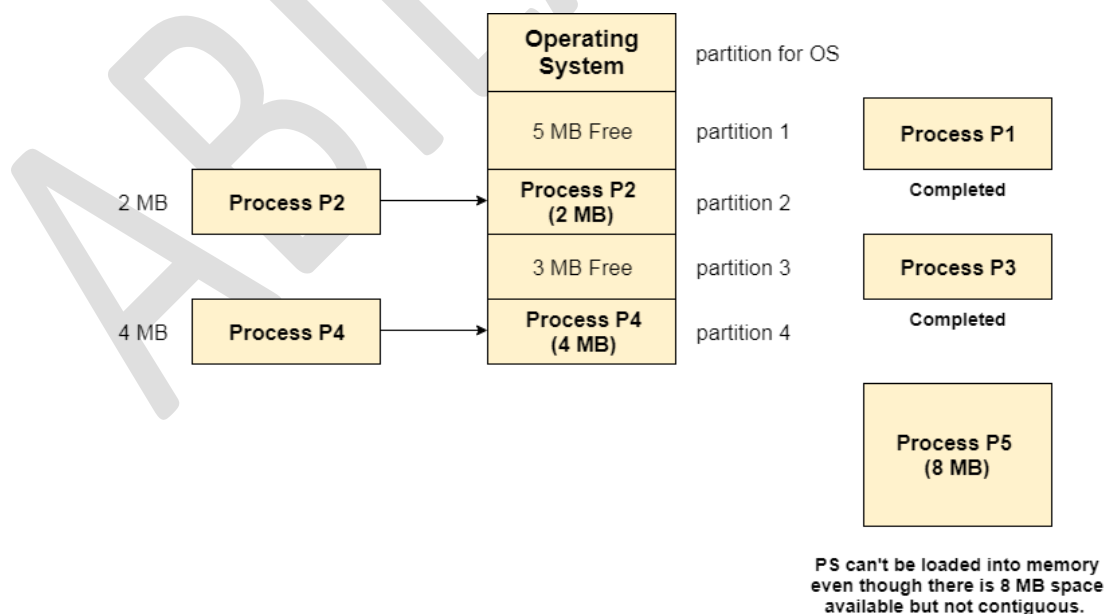
Disadvantages

1. External Fragmentation

Absence of internal fragmentation doesn't mean that there will not be external fragmentation.

Let's consider three processes P1 (1 MB) and P2 (3 MB) and P3 (1 MB) are being loaded in the respective partitions of the main memory. After some time P1 and P3 got completed and their assigned space is freed. Now there are two unused partitions (1 MB and 1 MB) available in the main memory but they cannot be used to load a 2 MB process in the memory since they are not contiguously located.

The rule says that the process must be contiguously present in the main memory to get executed. We need to change this rule to avoid external fragmentation.



External Fragmentation in Dynamic Partitioning

X

Paging

- In case of contiguous allocation scheme a process can be loaded into memory for execution if and only if contiguous memory is larger enough to hold the process
- Therefore external fragmentation is a common problem.
- One solution for this problem is compaction.
- The other solution to this problem could be to permit non contiguous address space so that a process can be allocated physical memory wherever it is present.
- This solution is implemented through the use of **paging scheme**.

Concept of paging:

- Paging is a memory-management scheme that permits the physical address space of a process to be non contiguous.
- Logical address space of a process can be noncontiguous; process is allocated physical memory whenever the latter is available
- Physical memory is divided into a fixed sized blocks called **frames**.
- Logical memory is divided into blocks of same/fixed size called **pages**.
- Allocation of main memory to processes for execution is then just mapping pages to frames.
- Allocates pages to frames in memory.
- Keep track of all free frames
- To run a program of size n pages, need to find n free frames and load program
- Set up a page table to translate logical to physical addresses

Address Translation Scheme

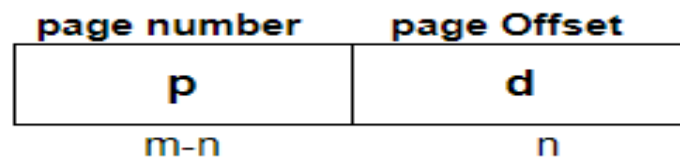
Address generated by CPU is divided into

- **Page number (P)** – used as an index into a *page table* which contains base address of each page in physical memory
- **Page offset (d)** – combined with base address to define the physical memory address that is sent to the memory unit

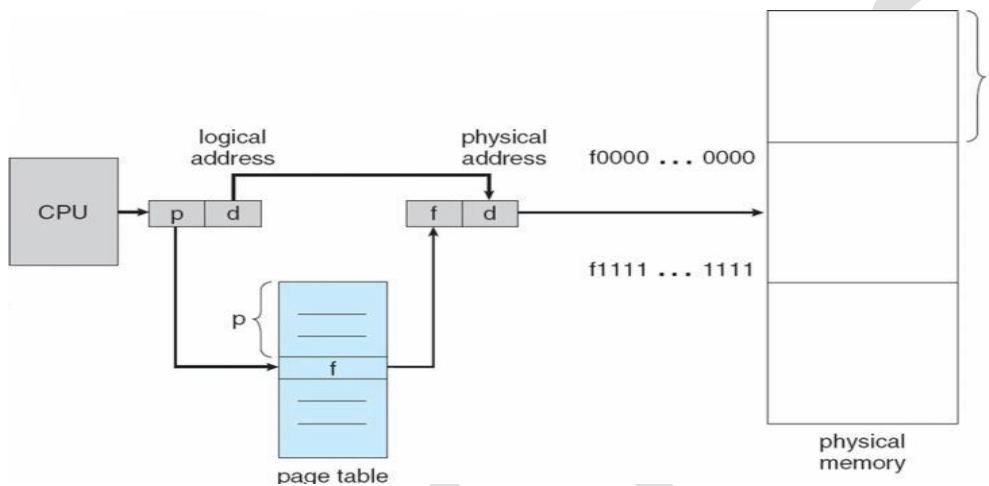
The Page size (like the frame size) is defined with the help of hardware. It is important to note here that the size of the page is typically the power of 2 that varies between 512 bytes and 16 MB per page and it mainly depends on the architecture of the computer.

If the size of **logical address space** is 2 raised to the power m and **page size** is 2 raised to the power n addressing units then the high order $m-n$ bits of **logical address designates the page number** and the n low-order **bits designate the page offset**.

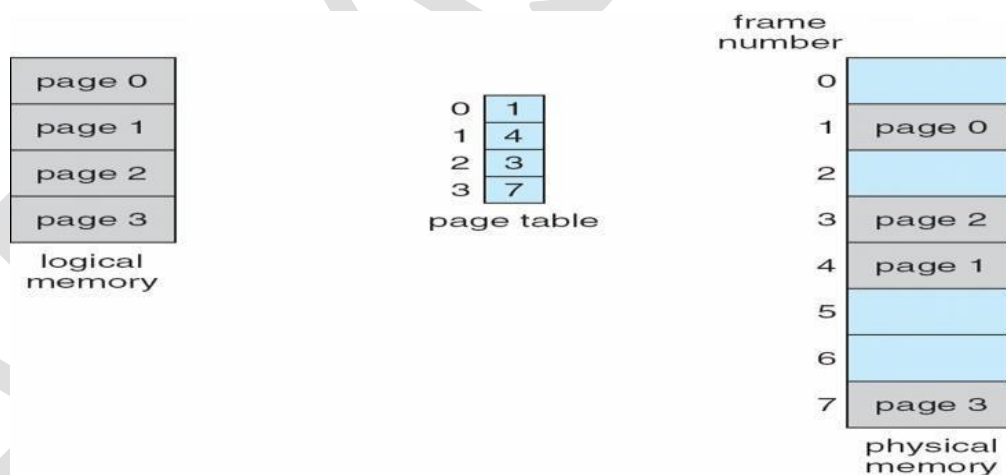
The logical address is as follows:



where **p** indicates the index into the page table, and **d** indicates the displacement within the page.



Paging Hardware



Paging Model of Logical and Physical Memory

Advantages:

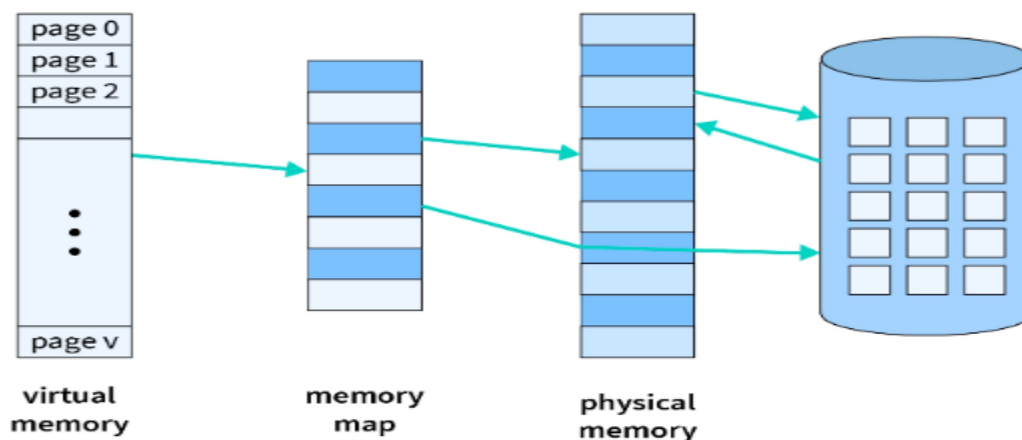
- It facilitates non contiguous allocation scheme
- Eliminates fragmentation(only external fragmentation)
- Higher degree of multiprogramming
- Increased memory and processor utilization
- Eliminates compaction overhead

Disadvantages:

- Page address mapping hardware increases the cost of the computer
- Memory must be used to store the various tables
- Processor time is utilized to update various tables
- Internal fragmentation do occur if page sizes are large
- Memory may still contain information that is not used

VIRTUAL MEMORY

- Virtual memory is a technique that allows the execution of process that may not be completely in memory
- Virtual Memory is a space where large programs can store themselves in the form of pages during their execution and only the required pages or portions of processes are loaded into the main memory.
- The main visible advantage of this scheme is that programs can be larger than physical memory.
- Virtual memory is the separation of user logical memory from physical memory this separation allows an extremely large virtual memory to be provided for programmers when only a smaller physical memory is available.



Concept of virtual memory

Advantages of Virtual Memory

- Virtual Memory allows you to run more applications at a time.
- With the help of virtual memory, you can easily fit many large programs into smaller programs.
- With the help of Virtual memory, a multiprogramming environment can be easily implemented.
- As more processes should be maintained in the main memory which leads to the effective utilization of the CPU.
- Data should be read from disk at the time when required.

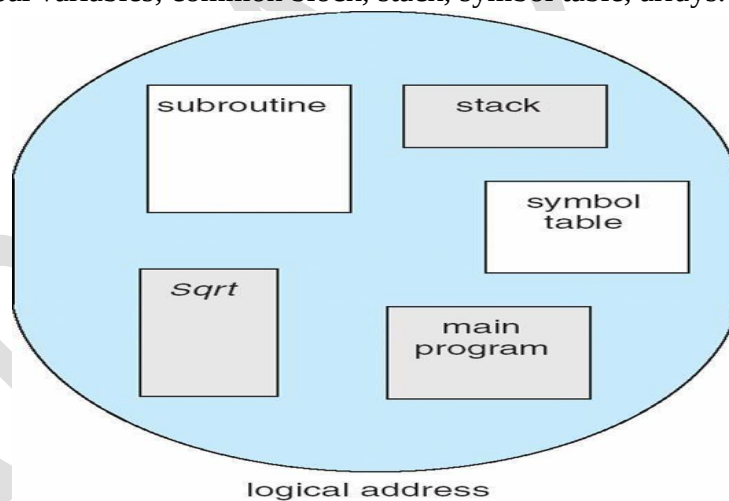
- Common data can be shared easily between memories.
- With the help of virtual memory, speed is gained when only a particular segment of the program is required for the execution of the program.
- The process may even become larger than all of the physical memory.

Disadvantages of Virtual Memory

- Virtual memory reduces the stability of the system.
- The performance of Virtual memory is not as good as that of RAM.
- If a system is using virtual memory then applications may run slower.
- Virtual memory negatively affects the overall performance of a system.
- It occupies the storage space, which might be otherwise used for long term data storage.
- This memory takes more time in order to switch between applications.

Segmented data

- Memory-management scheme that supports user view of memory.
- A program is a collection of segments.
- A segment is a logical unit such as: main program, procedure, function, method, object, local variable, global variables, common block, stack, symbol table, arrays.



Concept of segmentation

- Segmentation is a memory management scheme that supports user's view of main memory.
- The logical address space is collection of segment, each having name and length.
- The addresses specify both segment name and offset within the segment
- Since it is easy to work with numbers, segments are numbered.
- Thus logical address consists of two tuples: <segment number, offset>
- User programs when compiled reflect segments present in input.

Segmented architecture

- Logical address consists of a two tuple: <segment –number, offset>,
- Segment table-maps two-dimensional physical address; each table entry has:
- Base-contains the starting physical address where the segment reside in memory.
- Limit-specifies the length of the segment.
- The base corresponds to start physical address in memory, where as limit is the length of the segment.
- The segment number is used as an index into a segment table.
- The offset is logical address that lies between 0 and a limit specified in corresponding entry in the segment table
- If not the program is trying to access memory location which does not belong to segment & hence is trapped as an addressing error.
- If the offset is correct, the segment table gives base value to be added to offset to generate physical address.

Segmentation Hardware

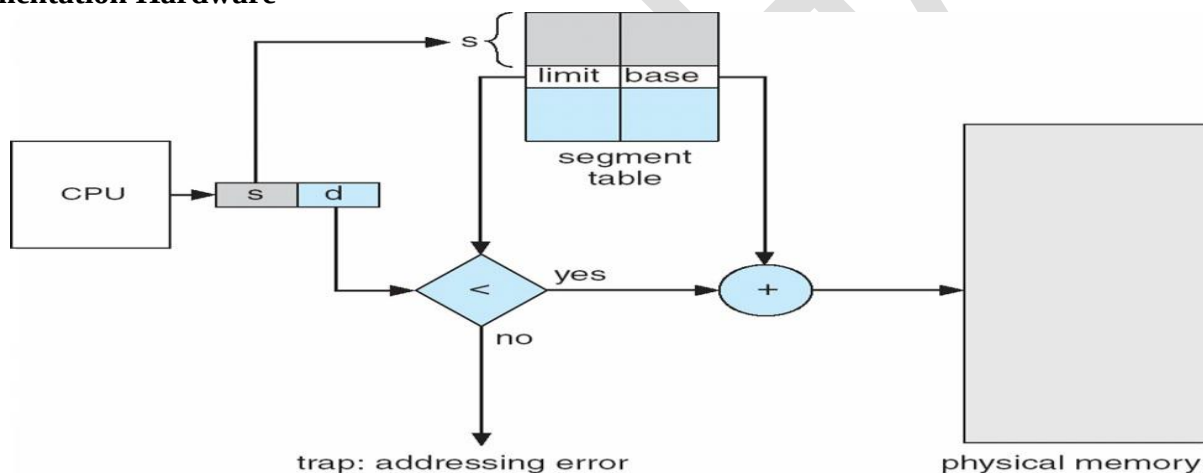
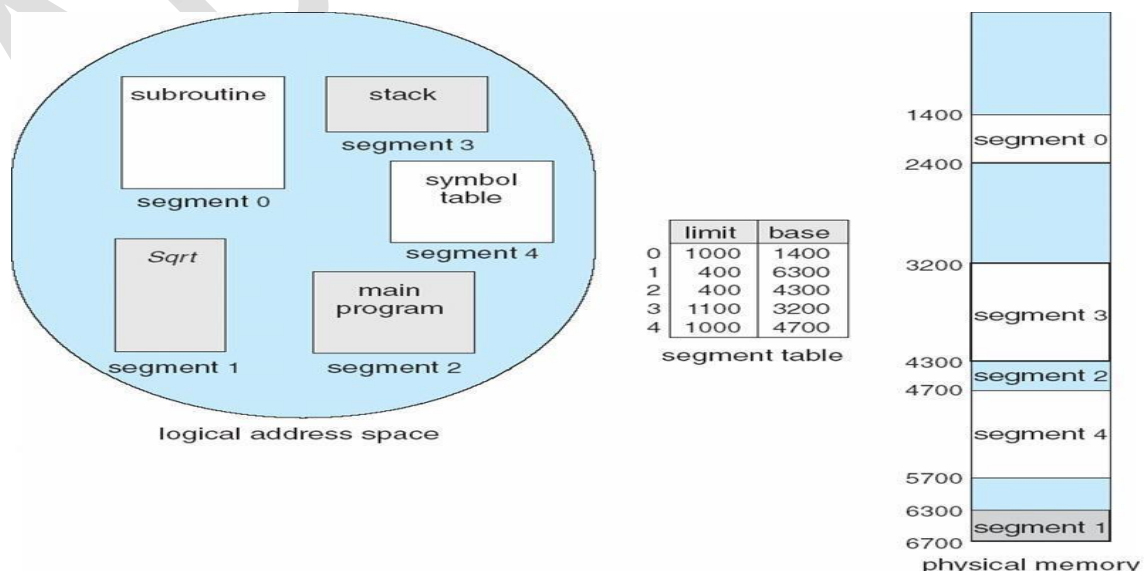


Illustration of segmentation



Segmented Paging

Pure segmentation is not very popular and not being used in many of the operating systems. However, Segmentation can be combined with Paging to get the best features out of both the techniques.

In Segmented Paging, the main memory is divided into variable size segments which are further divided into fixed size pages.

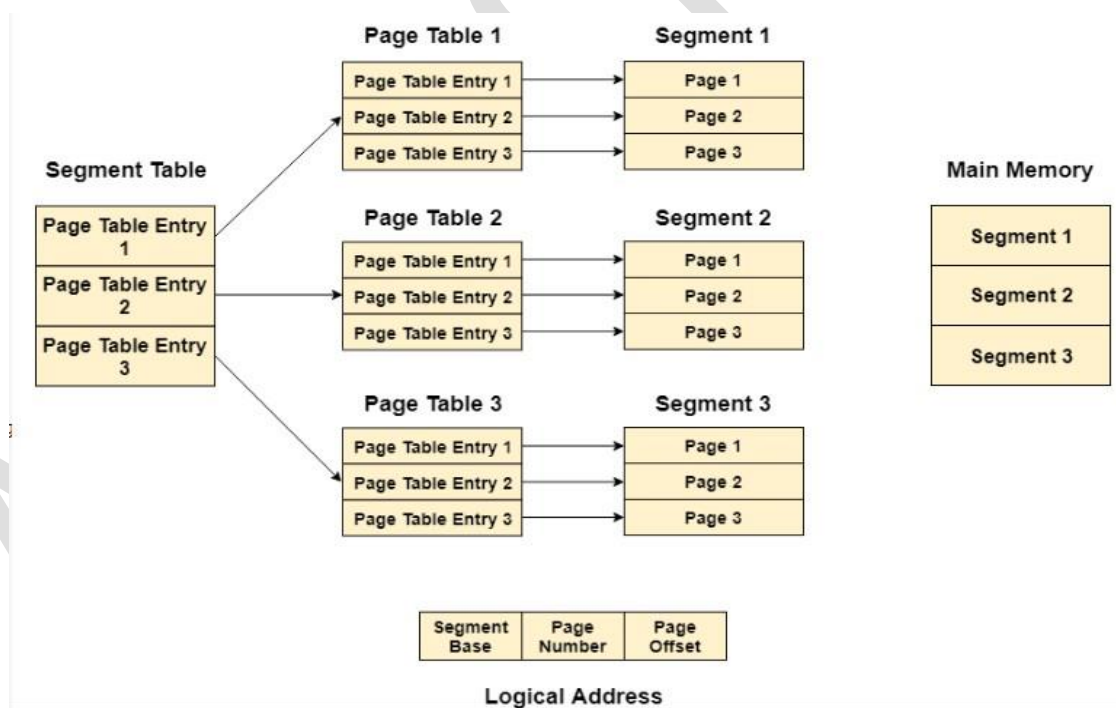
1. Pages are smaller than segments.
2. Each Segment has a page table which means every program has multiple page tables.
3. The logical address is represented as Segment Number (base address), Page number and page offset.

Segment Number → It points to the appropriate Segment Number.

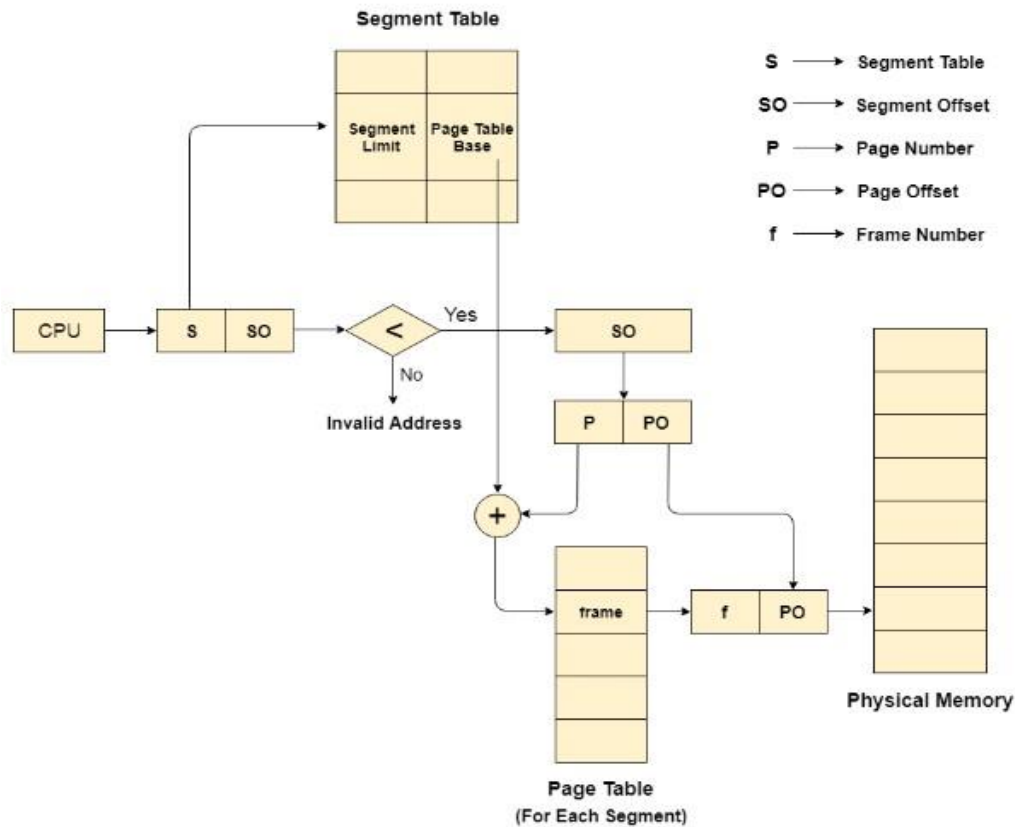
Page Number → It Points to the exact page within the segment

Page Offset → Used as an offset within the page frame

Each Page table contains the various information about every page of the segment. The Segment Table contains the information about every segment. Each segment table entry points to a page table entry and every page table entry is mapped to one of the page within a segment.



The CPU generates a logical address which is divided into two parts: Segment Number and Segment Offset. The Segment Offset must be less than the segment limit. Offset is further divided into Page number and Page Offset. To map the exact page number in the page table, the page number is added into the page table base. The actual frame number with the page offset is mapped to the main memory to get the desired word in the page of the certain segment of the process



Advantages of Segmented Paging

1. It reduces memory usage.
2. Page table size is limited by the segment size.
3. Segment table has only one entry corresponding to one actual segment.
4. External Fragmentation is not there.
5. It simplifies memory allocation.

Disadvantages of Segmented Paging

1. Internal Fragmentation will be there.
2. The complexity level will be much higher as compare to paging.
3. Page Tables need to be contiguously stored in the memory.

FILE SYSTEM

A file is a named collection of related information that is stored on secondary storage. A file usually represents programs (source and object) and data the information in a file is defined by its creator. Many different types of information may be stored in a file.

FILE ATTRIBUTES

The different types of attributes are listed below

- **Name:** The name can be in the human readable form.
- **Type:** This information is needed for those systems that support different types.
- **Location:** This information is used to a device and to the location of the file on that device.
- **Size:** This indicates the size of the file in bytes or words.
- **Protection:** This is access control information and controls reading, writing, executing and so on
- **Time, Date and User identifications:** the information about all files is kept in the directory structure that also resides on secondary storage.

File operations

1. Creating a file

- Two steps are necessary to create a file
- First, space must be found for the file in the system
- Second , an entry for the new file must be made in the directory
- The directory entry records the name of the file and the location in the system.

2. Writing a file

- To write a file given the name of the file, the system search the directory to find the location of the file.
- The system must keep the *write* pointer to the location in the file where the next write is to take place.
The write pointer must be updated whenever a write occurs.

3. Reading a file

- To read from a file, specifies the name of the file and directory is search for the associated directory entry.
- The system needs to keep *read* pointer to the location in the file where the next read is to take place.
- Once the read has taken place, read pointer is updated.

4. Repositioning with in a file

The directory is searched for the appropriate entry and the current file position is set to given value. This is also known as files seek.

5. Deleting a file

- To delete a file, we search the directory for the file name.
- If file is found in the directory entry, we release all file space and erase the directory entry.

6. Truncate a file

This function allows all attributes to remain unchanged (except for file length) but for the file to be reset to length zero.

7. Appending

Add new information to the end of an existing file.

8. Renaming

Give a new name to an existing file.

9. Open a file

If file need to be used, the first step is to open the file, using the *open* system call.

10. Close a file

Close is a system call used to terminate the use of an already used file.

File Types

A common technique for implementing file type is to include the type as part of the file name.

The name is split into two parts. 1) The name 2) An extension.

The system uses the extension to indicate the type of the file and the type of operations that can be done on that file.

Common File types

FILE TYPES	EXECUTION	FUNCTION
Source code	.Pas, .C, .P, etc	Source code in different Languages
Object code not Linked	.Obj, language,	Compiled machine
Executable program	.exe, .com, .bin language	Ready to run machine
Text	txt, doc	textual data, documents
Word processor	wp, text, doc	various word processor formats

- **Executable file-** In an executable file, the binary code that is loaded in the memory for execution is stored. It is stored in an exe type file.
- **Source file-** The source file has subroutines and functions that are compiled later.
- **Object file-** An object file is a sequence of bytes used by the linker.
- **Text file-** A text file is a sequence of characters.
- **Image file-** An image file is a sequence of visual information, for example, vector art.

Access Methods

Files contain a lot of information required by the system. The computer memory may require certain files during execution. We need very efficient methods to retrieve the information by accessing the files in the least time possible.

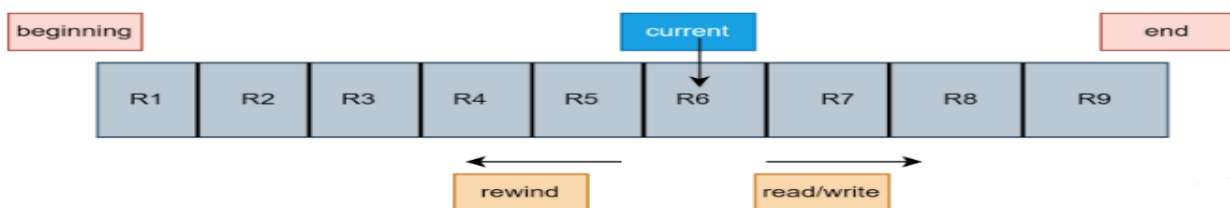
File access methods are needed to efficiently read and write data to and from files. Each method has its own advantages and disadvantages, and the choice of method depends on the specific needs of the application. Using an appropriate file access method can improve the performance and efficiency of an application.

The three types of file access methods are:

1 Sequential Access Method

It is one of the simplest access methods and is widely used in editors and compilers. The audio cassettes even they use the same access method.

The files are a collection of records. Accessing the file is equivalent to accessing the records. In the sequential access method, each record is accessed sequentially, one after the other. Consider the below image for more clarity.



Sequential Access Method

The figure represents a file. The current pointer is pointing to the record currently being accessed. In the sequential access method, the current pointer cannot directly jump to any record. It has to "cross" every record that comes in its path. Suppose there are nine records in the file from R1 to R9. The current pointer is at record R6. If we want to access record R8, we have to first access record R6 and record R7. This is one of the major disadvantages of the sequential access method.

Advantages

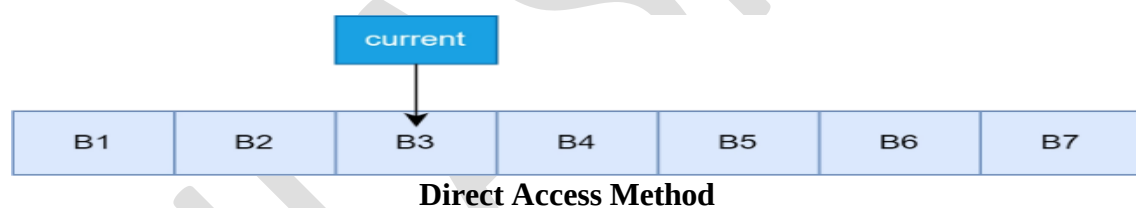
- Simple and easy to implement, requiring minimal hardware and software support.
- Low cost, as it doesn't require complex indexing or search algorithms.
- Highly efficient for dealing with large data sets, as data is stored in a linear fashion and can be accessed in a predictable manner.
- Suitable for applications that require processing of data in the order it was written, such as backup systems or data archival.
- Sequential access devices, such as magnetic tape, have a high storage capacity and can store data for long periods of time, making them suitable for archival storage.

Disadvantages

- Inefficient for random access: Sequential access is not efficient for randomly accessing data within a file, as it requires searching through the entire file to find the desired data.
- Limited concurrency: Concurrent access to a sequential file can be challenging, as only one process can access the file at a time.
- Inflexible data retrieval: Since data must be retrieved in the order it was stored, sequential access can be inflexible for applications that require accessing data in a non-linear fashion.
- Limited real-time access: Sequential access is not suitable for real-time systems, as data retrieval time can be unpredictable.

2. Direct Access Method

In the direct access method, the files are considered as a sequence of blocks or records, just like the disk was considered to be divided into equal-sized blocks. The benefit of this method is that we can access any block randomly. The direct access method is known as the relative access method. The exact block address is known only to the operating system. When a user wants to access a particular block, he/she provides the relative block number, which the operating system then uses to find the exact block address. This type of access method is used in database management systems.

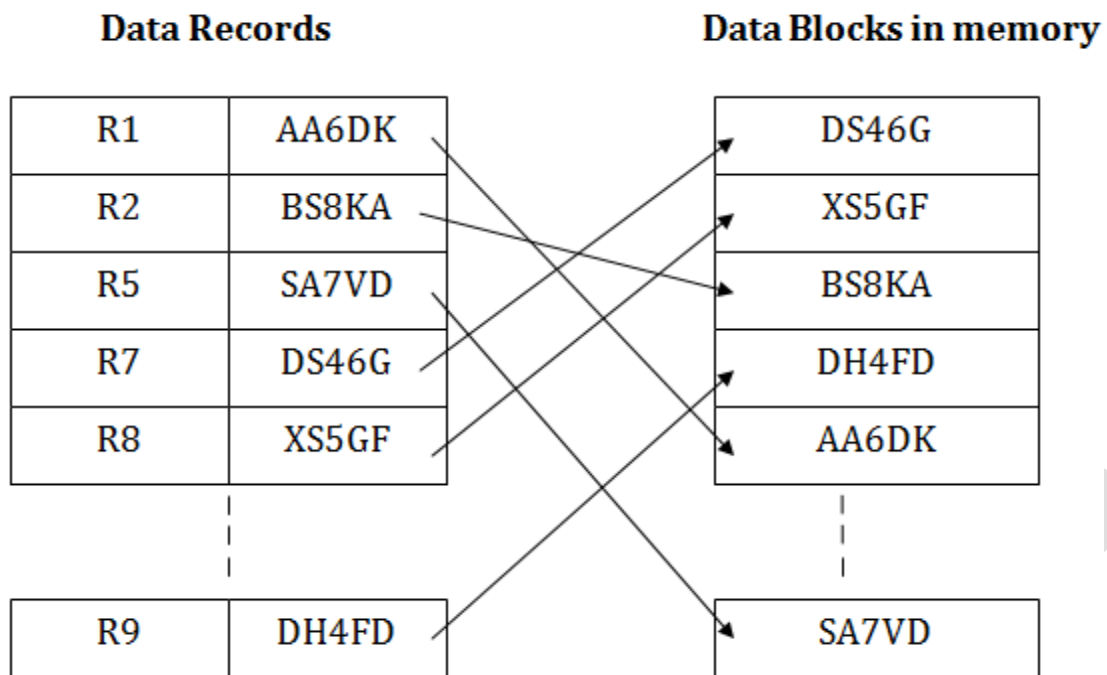


Advantages

- Random access to specific data within a file allowing for quick and efficient data retrieval.
- High concurrency, enabling multiple processes to access the same file simultaneously and efficiently share data.
- Flexible data retrieval, suitable for accessing data in a non-sequential fashion.
- Efficient data modification, as changes to specific data can be made without rewriting the entire file.
- Suitable for real-time systems that require fast and predictable data access.

3. Indexed sequential access method (ISAM)

ISAM method is an advanced sequential file organization. In this method, records are stored in the file using the primary key. An index value is generated for each primary key and mapped with the record. This index contains the address of the record in the file.



Indexed sequential access method (ISAM)

If any record has to be retrieved based on its index value, then the address of the data block is fetched and the record is retrieved from the memory.

Advantages

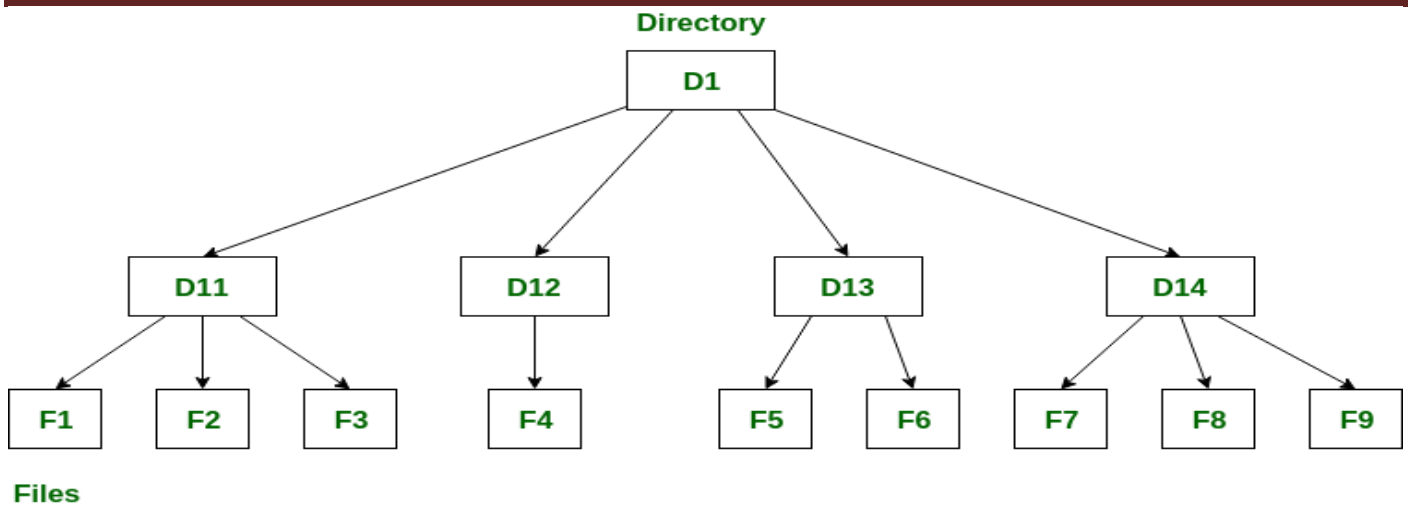
- In this method, each record has the address of its data block, searching a record in a huge database is quick and easy.
- This method supports range retrieval and partial retrieval of records. Since the index is based on the primary key values, we can retrieve the data for the given range of value. In the same way, the partial value can also be easily searched, i.e., the student name starting with 'JA' can be easily searched.

Disadvantages

- This method requires extra space in the disk to store the index value.
- When the new records are inserted, then these files have to be reconstructed to maintain the sequence.
- When the record is deleted, then the space used by it needs to be released. Otherwise, the performance of the database will slow down.

Directory Structures

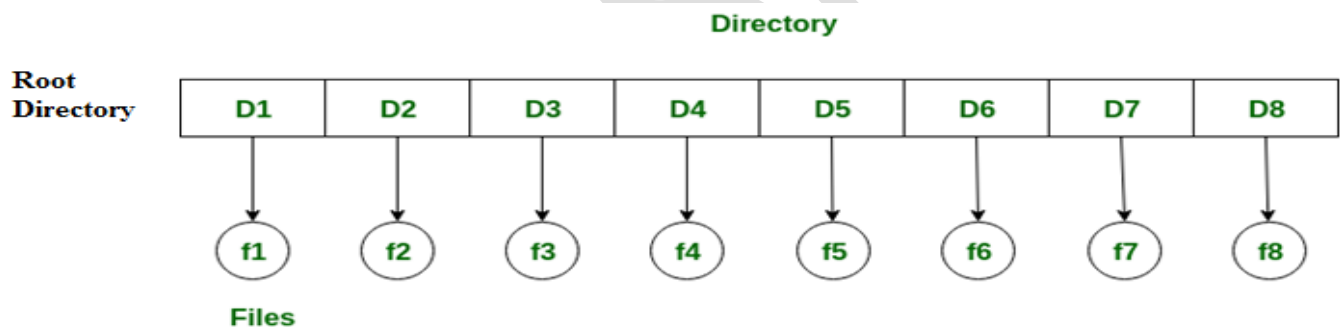
A **directory** is a container that is used to contain folders and files. It organizes files and folders in a hierarchical manner.



Directory structure

Single level directory

Single level directory structure has only one directory which is called the **root** directory. The users are not allowed to create subdirectories under the root directory. All the files created by the several users are present in the root directory only. As you can see in the below diagram all the file F1, F2, F3, F4F8 created by the different users are present at the root directory.



Single level directory

Advantages

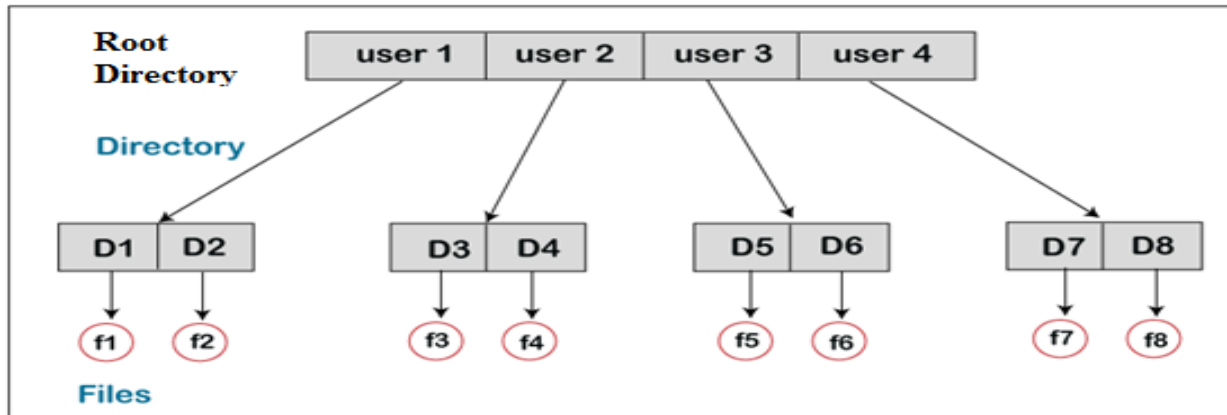
1. The implementation of a single-level directory is so easy.
2. In a single-level directory, if all the files have a small size, then due to this, the searching of the files will be easy.
3. In a single-Level directory, the operations such as searching, creation, deletion, and updating can be performed.

Disadvantages

1. If the size of the directory is large in Single-Level Directory, then the searching will be tough.
2. In a single-level directory, we cannot group the similar type of files.
3. There is a possibility of collision because the two files cannot have the same name.
4. The task of choosing the unique file name is a little bit complex.

Two level directory

In **Two-level directory structure**, the users create directory directly inside the root directory. But once a user creates such directory, further he cannot create any subdirectory inside that directory. Observe the figure below, 4 users have created their separate directory inside the root directory. But further, no subdirectory is created by the users.



Two level directory

Advantages

1. In the two-level directory, various users have the same file and also directory name.
2. Because of using the user-grouping and pathname, searching of files is quite easy.

Disadvantages

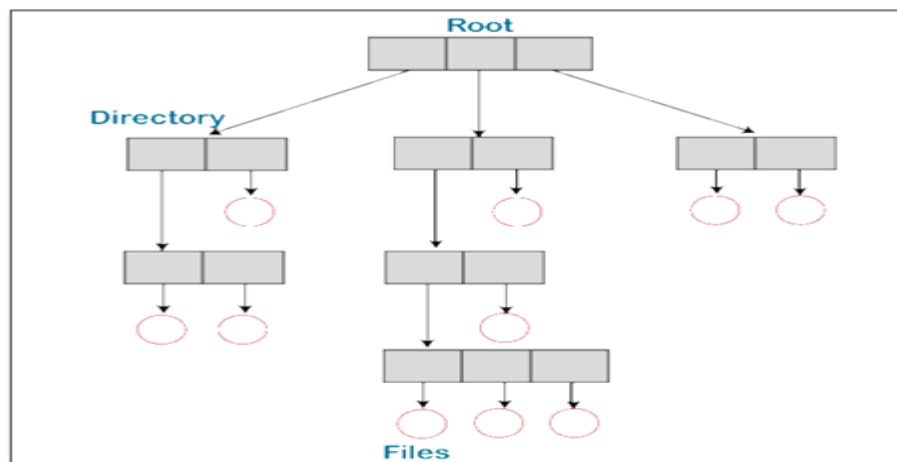
1. In a two-level directory, one user cannot share the file with another user.
2. Another disadvantage with the two-level directory is it is not scalable.

Tree structured directory

In a tree-structured directory, there is an own directory of each user, and any user is not allowed to enter into the directory of another user. Although the user can read the data of root, the user cannot modify or write it. The system administrator only has full access to the root directory. In this, searching is quite effective and we use the current working concept. This allows users to create their own subdirectories and to organize their files accordingly. Here the tree has a root directory. And every file in the system has a unique path name. A path name is the path from the root, through all the subdirectories to a specified file.

Here the path names can be of two types.

- An absolute path name begins at the root and follows a path down to the specified file, giving the directory name on the path. Ex:- root/spell/mail/prt/first.
- A relative pathname defines a path from the current directory ex:- prt/first is relative path name.



Tree structured directory

Advantages

1. The tree-structured directory is very scalable.
2. In the tree-structures directory, the chances of collision are less.
3. In the tree-structure directory, the searching is quite easy because, in this, we can use both types of paths, which are the absolute path and relative path.

Disadvantages

1. In the tree-structure directory, the files cannot be shared.
2. Tree-structure directory is not efficient because, in this, if we want to access a file, then it may go under multiple directories.
3. Another disadvantage of the tree-structure directory is that each file does not fit into the hierarchal model. We have to save the files into various directories.

Acyclic- graph directory

In the tree-structure directory, the same files cannot exist in the multiple directories, so sharing the files is the main problem in the tree-structure directory. With the help of the acyclicgraph directory, we can provide the sharing of files. In the acyclic-graph directory, more than one directory can point to a similar file or subdirectory. We can share those files among the two directory entries. With the help of aliases, and links, we can create this type of directory graph.

We may also have a different path for the same file. Links may be of two kinds, which are hard link (physical) and symbolic (logical) links.

If we delete the files in acyclic graph structures, then

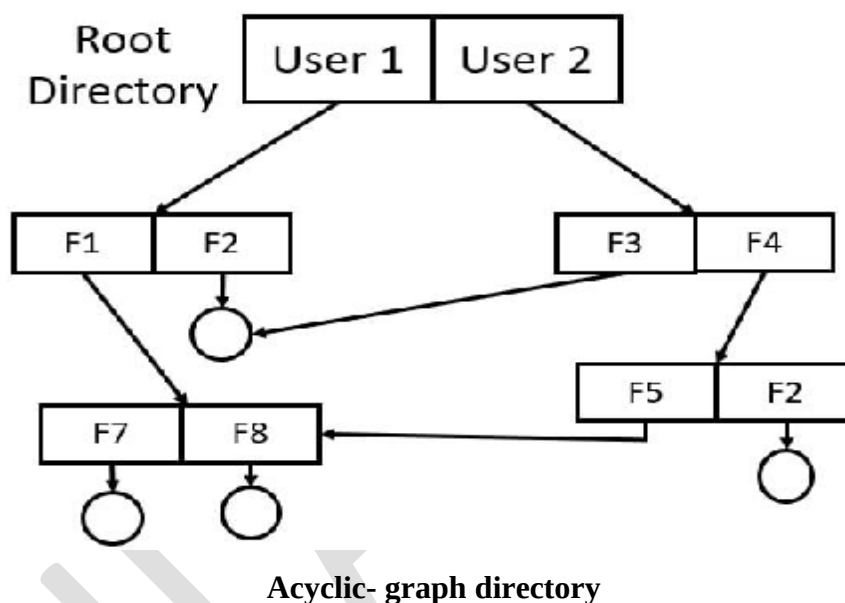
- In the hard link (physical) case, we can remove the actual files only if all the references to the file are deleted.
- In the symbolic link (logical) case, we just delete the file, and there is only a dangling point that is left.

Advantages

1. In the acyclic-graph directory, the sharing of files is possible.
2. In the acyclic-graph directory, because of different-different paths, searching is easy.

Disadvantages

1. If the files are shared through linking, there may be a problem in the case of deleting.
2. If we are using soft link, then in this case, if the file is deleted then there is only a dangling pointer which is left.
3. If we are using hard link, in this case, when we delete a file, then we also have to remove all the reference connected with it.



File Systems

File system is the part of the operating system which is responsible for file management. It provides a mechanism to store the data and access to the file contents including data and programs. Some Operating systems treats everything as a file for example Ubuntu.

The File system takes care of the following issues

- **File Structure:** We have seen various data structures in which the file can be stored. The task of the file system is to maintain an optimal file structure.
- **Recovering Free space:** Whenever a file gets deleted from the hard disk, there is a free space created in the disk. There can be many such spaces which need to be recovered in order to reallocate them to other files.
- **Disk space assignment to the files:** The major concern about the file is deciding where to store the files on the hard disk.

There are various disks scheduling algorithm which will be covered later in this tutorial.

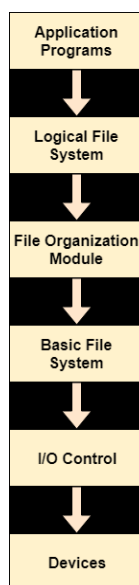
- **Tracking data location:** A File may or may not be stored within only one block. It can be stored in the non-contiguous blocks on the disk. We need to keep track of all the blocks on which the part of the files reside.

File System Structure

File System provide efficient access to the disk by allowing data to be stored, located and retrieved in a convenient way. A file System must be able to store the file, locate the file and retrieve the file. Most of the Operating Systems use layering approach for every task including file systems.

Every layer of the file system is responsible for some activities.

The image shown below, elaborates how the file system is divided in different layers, and also the functionality of each layer.



- When an application program asks for a file, the first request is directed to the logical file system. The logical file system contains the Meta data of the file and directory structure. If the application program doesn't have the required permissions of the file then this layer will throw an error. Logical file systems also verify the path to the file.
- Generally, files are divided into various logical blocks. Files are to be stored in the hard disk and to be retrieved from the hard disk. Hard disk is divided into various tracks and sectors. Therefore, in order to store and retrieve the files, the logical blocks need to be mapped to physical blocks. This mapping is done by File organization module. It is also responsible for free space management.
- Once File organization module decided which physical block the application program needs, it passes this information to basic file system. The basic file system is responsible for issuing the commands to I/O control in order to fetch those blocks.
- I/O controls contain the codes by using which it can access hard disk. These codes are known as device drivers. I/O controls are also responsible for handling interrupts.

FILE SYSTEM IMPLEMENTATION

File allocation methods

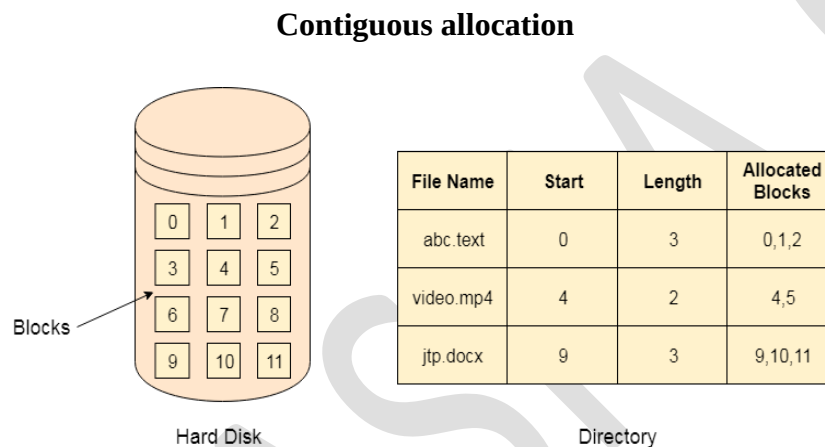
There are 3 major methods of allocating disk space.

1. Contiguous allocation

- The contiguous allocation method requires each file to occupy a set of contiguous block on the disk.
- Contiguous allocation of a file is defined by the disk address and length of the first block.

If the file is 'n' block long and starts at location 'b', then it occupies blocks b,b+1,b+2,...,b+n-1.

- The directory entry for each file indicates the address of the starting block and length of the area allocated for this file.



Contiguous Allocation

- Contiguous allocation of file is very easy to access.
- For the sequential access, the file system remembers the disk address of the last block referenced and, when necessary read next block.
- For *direct access* to block 'i' of a file that starts at block 'b', we can immediately access block b+i.
- Thus both sequential and direct access can be supported by contiguous allocation.

Advantages

- It is simple to implement.
- We will get Excellent read performance.
- Supports Random Access into files.

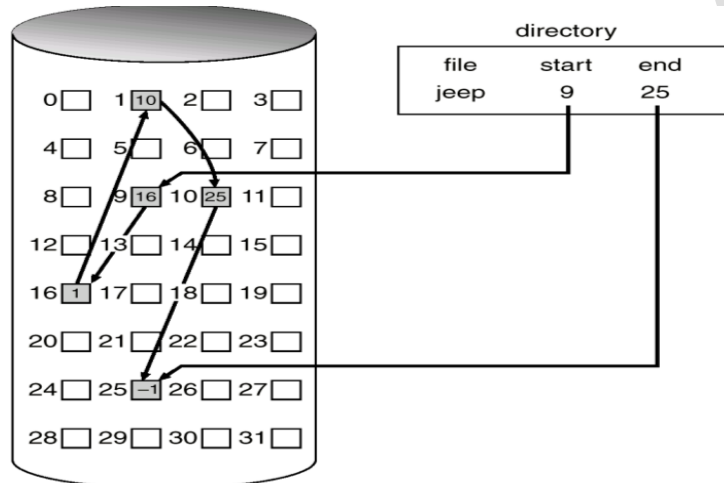
Disadvantages

- The disk will become fragmented.
- It may be difficult to have a file grow.

2. Linked allocation

- Linked allocation solves all the problems of contiguous allocation.
- With linked allocation, each file is a linked list of disk blocks, the disk block may be scattered anywhere on the disk.
- The directory contains a pointer to the first and last blocks of the file

Ex:-A file have five blocks start at block 9, continue at block 16, then block 1, block 10 and finally block 25. Each block contains a pointer to the next block.



The major problem is that it can be used only for sequential access-files.

- To find the I^{th} block of a file, we must start at the beginning of that file, and follow the pointers until we get to the I^{th} block.
- It cannot support the direct access.
- It requires space for the pointers. If a pointer requires 4 bytes out of 512 byte block, then 0.78% of disk is being used for pointers, rather than for information.
- The solution to this problem is to allocate blocks in to multiples, called clusters and to allocate the clusters rather than blocks.
- Another problem is reliability. The files are linked together by pointers scattered all over the disk what happen if a pointer were lost or damaged.

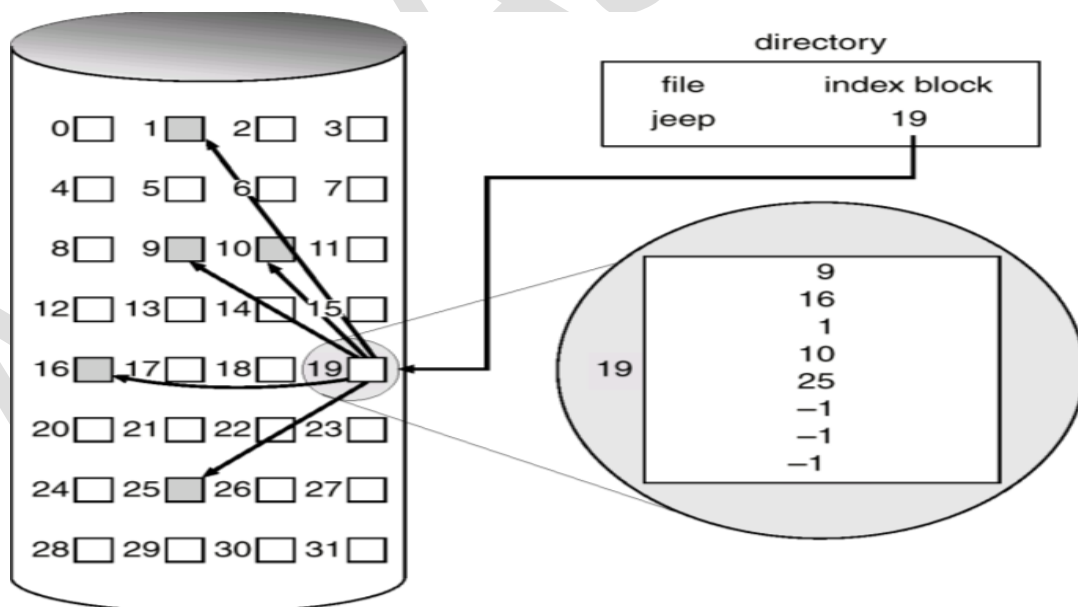
FAT (file allocation table)

- An important variation on the linked allocation method is the use of a file allocation table.
- The table has one entry for each disk block, and is indexed by block number. The FAT is used much as a linked list.
- The directory entry contains the block number of the first block of the file.

- The table entry contains the block number then contains the block number of the next block in the file.
- This chain continuous until the last block, which has a special end of file values as the table entry. Unused blocks are indicated by a '0' table value.
- Allocation a new block to a file is a simple.
- First finding the first 0-value table entry, and replacing the previously end of file value with the address of the new block. The 0 is then replaced with end of file value.

3. Indexed allocations

- Linked allocation solves the external fragmentation and size declaration problems of contiguous allocation.
- However in the absence of a FAT, linked allocation cannot support efficient direct access.
- The pointers to the blocks are scattered with the blocks themselves all over the disk and need to be retrieved in order.
- Indexed allocation solves this problem by bringing all the pointers together in to one location i.e **the index block**.
- Each file has its own index block, which is an array of disk block addresses.
- The I^{th} entry in the index block points to the I^{th} block of the file.
- The directory contains the address of the index block.



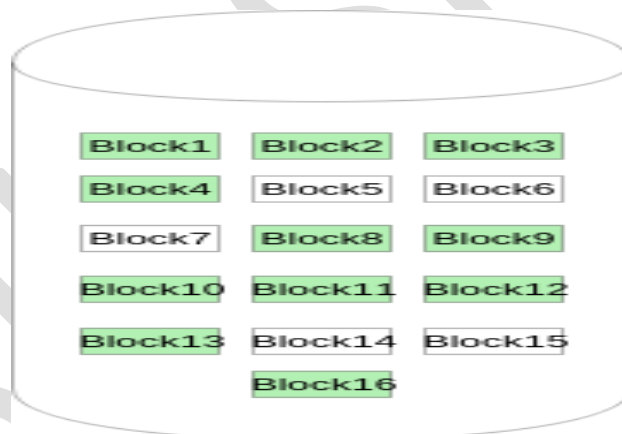
- To read the i^{th} block use the pointer in the i^{th} index block entry to find and read the desired block.
- When the file is created, all pointers in the index block are set to nil.
- When the i^{th} block is first written, a block is obtained from the free space manager, and its address is put in the i^{th} index block entry.

- It supports the direct access without suffering from external fragmentation.
- But it suffers from the wasted space. The pointer overhead of the index block is generally greater than the pointer overhead of linked allocation.

FREE SPACE MANAGEMENT

- To keep track of free disk space, the system maintains a free space list. The free space list records all disk blocks that are free.
- To create a file we search the free space list for the required amount of space, and allocate that space to the new file.
- This space is then removed from the free space list.
- When the file is deleted, its disk space is added to the free space list.

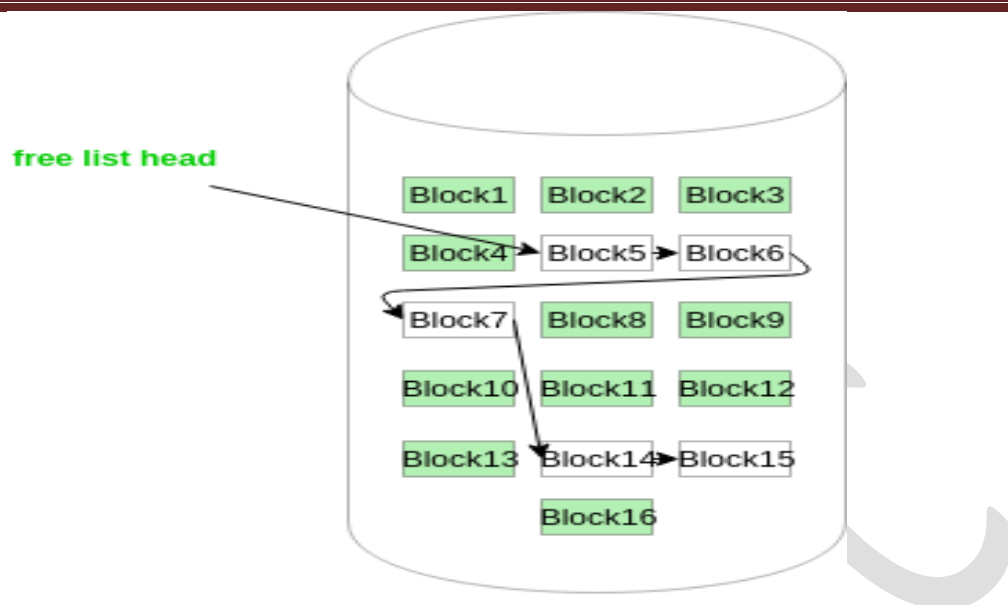
1. Bitmap or Bit vector – A Bitmap or Bit Vector is series or collection of bits where each bit corresponds to a disk block. The bit can take two values: 0 and 1: 0 indicates that the block is allocated and 1 indicates a free block. The given instance of disk blocks on the disk in Figure 1 (where green blocks are allocated) can be represented by a bitmap of 16 bits as: **0000111000000110**.



Advantages

- Simple to understand.
- Finding the first free block is efficient. It requires scanning the words (a group of 8 bits) in a bitmap for a non-zero word. (A 0-valued word has all bits 0). The first free block is then found by scanning for the first 1 bit in the non-zero word.

2. Linked List – In this approach, the free disk blocks are linked together i.e. a free block contains a pointer to the next free block. The block number of the very first disk block is stored at a separate location on disk and is also cached in memory. In Figure-2, the free space list head points to Block 5 which points to Block 6, the next free block and so on. The last free block would contain a null pointer indicating the end of free list. A drawback of this method is the I/O required for free space list traversal.



3. **Grouping-** This approach stores the address of the free blocks in the first free block. The first free block stores the address of some, say n free blocks. Out of these n blocks, the first $n-1$ blocks are actually free and the last block contains the address of next free n blocks. An **advantage** of this approach is that the addresses of a group of free disk blocks can be found easily.
4. **Counting** – This approach stores the address of the first free disk block and a number n of free contiguous disk blocks that follow the first block. Every entry in the list would contain:
- Address of first free disk block
 - A number n

Here are some advantages and disadvantages of free space management techniques in operating systems:

Advantages:

1. **Efficient use of storage space:** Free space management techniques help to optimize the use of storage space on the hard disk or other secondary storage devices.
2. **Easy to implement:** Some techniques, such as linked allocation, are simple to implement and require less overhead in terms of processing and memory resources.
3. **Faster access to files:** Techniques such as contiguous allocation can help to reduce disk fragmentation and improve access time to files.

Disadvantages:

1. **Fragmentation:** Techniques such as linked allocation can lead to fragmentation of disk space, which can decrease the efficiency of storage devices.
2. **Overhead:** Some techniques, such as indexed allocation, require additional overhead in terms of memory and processing resources to maintain index blocks.
3. **Limited scalability:** Some techniques, such as FAT, have limited scalability in terms of the number of files that can be stored on the disk.

4. Risk of data loss: In some cases, such as with contiguous allocation, if a file becomes corrupted or damaged, it may be difficult to recover the data.
5. Overall, the choice of free space management technique depends on the specific requirements of the operating system and the storage devices being used. While some techniques may offer advantages in terms of efficiency and speed, they may also have limitations and drawbacks that need to be considered.

ABBLASHA.C

Introduction to Unix System:

UNIX is a portable, multitasking, multiuser, time sharing operating system(OS).

Founder of UNIX: The UNIX was founded by Ken Thompson, Dennis Ritchie and Brian kerninghan at AT & T Bell Labs research in 1969

Need of UNIX: With other OS, additional software must be purchased for networking but with UNIX, network capability is the part of the operating system.

History of UNIX:

1960 Bell labs involved in the project with MIT, General Electric and Bell Laboratories to develop a time sharing system called MULTICS (Multiplexed Operating and Computing System). 1969 Ken Thompson wrote the first version of the UNIX called UNICS (Uniplexed Information and Computing System) 1970 Finally UNICS became UNIX.

Differences between UNIX and Windows:

UNIX	Windows
UNIX is an open source(free download)	Windows is not an open source(paid one)
UNIX has very high security system	Windows has low security system
UNIX is a command based operating system	Windows is not a command based operating system
The file system is arranged in hierarchical manner	The file system is arranged in parallel manner
UNIX is not a user friendly	Windows is a user friendly
Free office(such as excel, PowerPoint and others)	Pay for MS Office
Low Hardware cost	High Hardware cost
Customizable add features	Not customizable

Advantages of UNIX

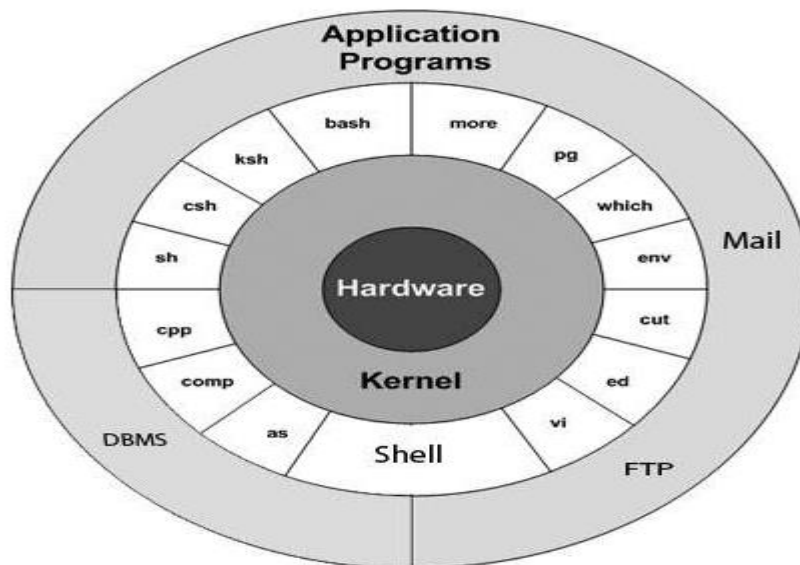
1. **Stability:** UNIX is known for its stability and reliability. It can run for long periods of time without requiring a reboot, which makes it ideal for critical systems that need to run continuously.
2. **Security:** UNIX has a robust security model that includes file permissions, user accounts, and network security features. This makes it a popular choice for systems that require high levels of security.
3. **Scalability:** UNIX can be scaled up to handle large workloads and can be used on a variety of hardware platforms.
4. **Flexibility:** UNIX is highly customizable and can be configured to suit a wide range of needs. It can be used for everything from simple desktop systems to complex server environments.

5. **Command-line interface:** UNIX's command-line interface allows for powerful and efficient interaction with the system.

Disadvantages of UNIX

1. **Complexity:** UNIX can be complex and difficult to learn for users who are used to graphical user interfaces ([GUIs](#)).
2. **Cost:** Some UNIX systems can be expensive, especially when compared to open-source alternatives like Linux.
3. **Lack of standardization:** There are many different versions of UNIX, which can make it difficult to ensure compatibility between different systems.
4. **Limited software availability:** Some specialized software may not be available for UNIX systems.
5. **Steep learning curve:** UNIX requires a certain level of technical knowledge and expertise, which can make it challenging for novice users.

Architecture of UNIX System/ UNIX System Structure:



The Architecture of the UNIX system is divided into 4 major components. They are:

- 1) The Kernel
- 2) The Shell
- 3) Files and Processes
- 4) System Calls

1. The Kernel

- The Kernel is the heart of the Operating System.
- It interface between Shell and Hardware.
- It performs Low Level Task.
- Eg: Device Management, Memory Management etc.

2. The Shell

- The Shell is a collection of UNIX Commands.
- The Shell acts as an interface between the user and the kernel.
- The Shell is a Command Line Interpreter(CLI)–Translates the commands provided by the user and converts it into a language that is understood by the Kernel.
- Only one Kernel running on the system, but several shells in action-one for each user who is logged in.
- Eg: C Shell, Bourne Shell, Korn Shell etc.

3. Files and Processes

- A File is an array of bytes and can contain anything.
- All the data in UNIX is organized into files.
- A Process is a program file under execution.
- Files and Processes belong to a separate hierarchical structure.

4. System Calls

- UNIX is written in C.
- Thousands of commands in the system uses a handful of functions called System Calls to communicate with the kernel.

Shell Programming:

The shell provides you with an interface to the UNIX system. It gathers input from you and executes programs based on that input. When a program finishes executing, it displays that program's output. A shell is an environment in which we can run our commands, programs, and shell scripts. There are different flavors of shells, just as there are different flavors of operating systems. Each flavor of shell has its own set of recognized commands and functions.

Vi editor:

The vi editor is the most popular editor in linux. The current version is really "vim", but to invoke it simply type "vi". The text format that vi uses is the most plain elementary text format that exists, the ascii format. But files created by vi can be run through other programs that convert the files to other formats, if desired. Open a terminal window in your linux system and click on it to make sure it is active. You will do all your work in the terminal window. If you are not already familiar with how to open a terminal window, find what directory you are in, list the files in a directory, move files, copy files, rename files, execute programs, add missing commands and linux command line operations in a terminal window you should first read my article how to use linux terminal.

Syntax: vi name_of_the_file [Here we are using the name "textvi"]

Once the file is open, you can move around by using cursors or the keys h, L and/(in case the keyboard does not have any arrow cursors).

Vi modes

Vi has three operating modes:

Regular mode: This is the mode you enter whenever you open a file. This mode allows typing commands

Insertion mode: This mode makes it possible to insert characters you capture inside of the document. To switch to insertion mode, just press the key insert on your keyboard or, by default, the key/

Replacement mode: This mode allows you to replace existing text by the text you capture. Just hit again to go to replacement mode and hit the key Esc to return to regular mode.

Command	Description
i	insert mode, (ESC to exit insert mode) allows text to be entered on the screen
a	Append to right mode
/word	Move to the occurrence of "word"
n	Locate the next occurrence
w	Advance to the next word
e	Advance to the next end of a word
b	Move to the previous word
3b	Move backward 3 words
yy	Copy line (then move cursor and use p to paste after cursor line)
dd	delete line
3dd	delete 3 lines
d	Delete remainder of a line
dw	Delete word
x	Delete character
o	Open space for new line below the cursor line
O	Open a line above the cursor
Ctrl-w	Move back a word in append mode

Starting vi:

You may use vi to open an already existing file by typing- **vi filename**

where "filename" is the name of the existing file. If the file is not in your current directory, you must use the full pathname.

Or you may create a new file by typing- **vi newname**

where "newname" is the name you wish to give the new file.

To open a new file called "testvi," enter- **vi testvi**

On-screen, you will see blank lines, each with a tilde (~) at the left, and a line at the bottom giving the name and status of the new file: **"testvi" [New file]**

Entering Text

In order to begin entering text in this empty file, you must change from command mode to insert mode. To do this, type: **i** or **press insert key**

Nothing appears to change, but you are now in insert mode and can begin typing text. In general, vi's commands do not display on the screen and do not require the Return key to be pressed.

Type a few short lines and press <Return> at the end of each line. If you type a long line, you will notice the vi does not word wrap, it merely breaks the line unceremoniously at the edge of the screen.

If you make a mistake, pressing <Backspace> or <Delete> may remove the error, depending on your terminal type.

Closing and Saving a File

With vi, you edit a copy of the file, rather than the original file. Changes are made to the original only when you save your edits.

To save the file and quit vi, type: ZZ

The vi editor is built on an earlier Unix text editor called ex. ex commands can be used within vi. ex commands begin with a (colon) and end with a <Return>. The command is displayed on the status line as you type. Some ex commands are useful when saving and closing files.

To save the edits you have made, but leave vi running and your file open:

1. Press <Esc>
2. Type :w
3. Press <Return>

To quit vi, and discard any changes you have made since last saving:

1. Press <Esc>
2. Type :q!
3. Press <Return>.

To save and quit:

1. Press <Esc>.
2. Type : wq
3. Press <Return>

To run the saved file :

1. Press <Esc>.
2. Type :wq
3. Press <Return>
4. Type: sh textvi

This will run the codes written

Shell Types:

In UNIX there are two major types of shells:

1. The Bourne shell. If you are using a Bourne-type shell, the default prompt is the \$ character.
2. The C shell. If you are using a C-type shell, the default prompt is the % character.

There are again various subcategories for Bourne Shell which are listed as follows:

- Bourne shell (sh)
- Korn shell (ksh)
- Bourne Again shell (bash)
- POSIX shell (sh)

The different C-type shells follow:

- C shell (csh)
- TENEX/TOPS C shell (tcsh)

The original UNIX shell was written in the mid-1970s by Stephen R. Bourne while he was at AT&T Bell Labs in New Jersey. The Bourne shell was the first shell to appear on UNIX systems, thus it is referred to as "the shell". The Bourne shell is usually installed as /bin/sh on most versions of UNIX. For this reason, it is the shell of choice for writing scripts to use on several different versions of UNIX.

Shell Command line processing:

Shell command-line processing in Unix involves multiple steps to interpret, execute, and return the output of a command. The shell acts as an interface between the user and the Unix operating system.

Steps in Command Line Processing

When a command is entered in the shell, the following steps occur:

1. **Reading the Command:** The shell reads the entire command entered by the user.
2. **Tokenization (Parsing the Command):** The command is broken into tokens (command, options, arguments).
3. **Variable and Wildcard Expansion:** Shell expands variables (\$HOME), wildcards (*), and special characters.
4. **Command Substitution:** Commands within backticks (`...`) or \$(...) are executed first.
5. **Redirection and Piping:** Input (<), output (>), and pipes (|) are processed.
6. **Command Execution:** The shell determines if it is a **built-in** or an **external** command and executes it.
7. **Process Completion:** The command runs, and the shell waits for completion.
8. **Exit Status Return:** Every command returns an exit status (0 for success, non-zero for errors).

Shell Script features of UNIX:

- 1) **Simple design, organization and functioning:** The architecture of the UNIX is simple, organized and functional.
- 2) **Portability:** The code can be changed and compiled on a new machine.
- 3) **UNIX Shell:** The user interface UNIX Shell provides the services that the user wants.
- 4) **Hierarchical file system:** UNIX uses a hierarchical file structure to store information.
- 5) **Multi user:** UNIX allows more than one user to share the same computer system at the same time.
- 6) **Multi-tasking:** More than one program can be run at a time.
- 7) **Security:** UNIX provides high security level(System level security and File level security)
- 8) **Pipes and Filters:** In UNIX, we can create complex programs from simple programs.
- 9) **Utilities:** UNIX has over 200 utility programs for various functions.
- 10) **Machine Independence:** The system hides the machine architecture from the user, making it easier to write applications that can run any computer system.

System variables:

Variable	Description
\$HOME	The home directory of the current user
\$USER	The current logged-in user
\$PATH	Directories searched for executable commands
\$SHELL	The shell being used (e.g., /bin/bash)
\$PWD	The present working directory
\$LOGNAME	The login name of the user

\$HOSTNAME	The system's hostname
\$LANG	The language setting of the system
\$TERM	The terminal type
\$EDITOR	The default text editor
\$UID	The user ID

Ex:

To print the value of a system variable, use echo:

```
echo $HOME
```

```
echo $USER
```

```
echo $PATH
```

User defined variable:

Variables are defined as follows::

```
variable_name = variable_value
```

For example: NAME = "sscas"

Above example defines the variable NAME and assigns it the value "sscas". Variables of this type are called scalar variables. A scalar variable can hold only one value at a time.

The shell enables you to store any value you want in a variable. For example: VAR1="ssczsc "

```
VAR2=100
```

Shell Screen Interface:

In Unix, the **shell screen interface** refers to the environment where users interact with the operating system through a command-line interface (CLI). Here's a breakdown of key aspects:

1. Shell

The shell is a command-line interpreter that processes commands and returns output. Common Unix shells include:

- **Bash (Bourne Again Shell)** – Most commonly used.
- **Zsh (Z Shell)** – An extended version of Bash with more features.
- **Ksh (Korn Shell)** – An improved Bourne shell.
- **Tcsh (TENEX C Shell)** – An enhanced version of the C shell.

2. Terminal Emulator

A **terminal emulator** provides a graphical interface to interact with the shell. Popular terminal emulators include:

- **xterm**
- **gnome-terminal**
- **konsole**
- **alacritty**
- **tmux** (for terminal multiplexing)

3. Screen Multiplexers

These tools allow multiple shell sessions within a single terminal:

- **GNU Screen** – Allows multiple terminal windows and session persistence.
- **Tmux (Terminal Multiplexer)** – Similar to Screen but more modern and customizable.

4. Basic Shell Screen Commands

Here are some essential commands for working within a Unix shell:

- **clear** – Clears the screen.
- **echo \$TERM** – Shows the current terminal type.
- **tput clear** – Clears the screen using terminal capabilities.
- **screen** – Starts a new screen session.
- **screen -r** – Reattaches to a detached screen session.

Read and Echo statement:

In Unix, **read** is used for taking user input, and **echo** is used for displaying output. Let's look at each in detail with examples.

1. read Command (User Input)

The **read** command allows you to take input from the user.

Syntax: **read variable_name**

This waits for user input and stores it in **variable_name**.

Example 1: Simple Input

```
echo "Enter your name:"  
read name  
echo "Hello, $name!"
```

Output:

```
Enter your name:  
Alice  
Hello, Alice!
```

Example 2: Reading Multiple Inputs

You can take multiple inputs in one line.

```
echo "Enter your first name and last name:"  
read first last  
echo "Hello, $first $last!"
```

Output:

```
Enter your first name and last name:  
John Doe  
Hello, John Doe!
```

2. echo Command (Output)

The echo command prints text to the screen.

Syntax: `echo [options] "text"`

Example 1: Simple Output

```
echo "Hello, World!"
```

Output:

```
CopyEdit  
Hello, World!
```

Example 2: Displaying Variables

```
name="Alice"  
echo "Hello, $name!"
```

Output:

Hello, Alice!

Command Substitution in Unix

Command substitution allows the output of one command to be used as an argument in another command. This helps automate tasks by dynamically inserting command outputs.

Syntax of Command Substitution:

There are two ways to perform command substitution:

1. Using Backticks (`command`)

```
output=`command`
```

2. Using \$() (Recommended)

```
output=$(command)
```

Note: The \$() syntax is preferred because:

- It is more readable.
- It allows **nesting** (using command substitution within another command).

Example: Storing Command Output in a Variable

```
current_date=$(date)
```

```
echo "Today's date is: $current_date"
```

Output:

Today's date is: Sun Feb 2 12:45:30 UTC 2025

Escape Sequence Characters:

Escape sequence characters in Unix are special characters that are interpreted differently by the shell. They are commonly used in echo, printf, and other shell commands.

Common Escape Sequences

Escape Sequence	Description	Example
\n	New line	echo -e "Hello\n World"
\t	Horizontal tab	echo -e "Hello\t World"
\v	Vertical tab	echo -e "Hello\v World"
\b	Backspace	echo -e "Helloo \b World"
\r	Carriage return (overwrite line)	echo -e "Hello\r World"

\f	Form feed (page break)	echo -e "Hello\f World"
\a	Alert (beep sound)	echo -e "\a"
\\	Backslash	echo -e "C:\\Users\\Admin"
\'	Single quote	echo -e "It\'s a test"
\"	Double quote	echo -e "Hello \"World\""

Example 1: New Line (\n)

```
echo -e "Hello \n World"
```

Output:

```
Hello
World
```

Example 2: Tab (\t)

```
echo -e "Hello\tWorld"
```

Output:

```
Hello  World
```

Example 3: Backspace (\b)

```
echo -e "Hello\b World"
```

Output:

```
Hello World
```

(Deletes the extra "o" before printing.)

Example 4: Carriage Return (\r)

```
echo -e "Hello\rWorld"
```

Output:

```
World
```

(The word "Hello" is overwritten by "World".)

Shell Script Arguments in Unix

Shell script arguments allow passing inputs to a script when executing it. These arguments are accessed within the script using special variables.

Accessing Arguments

Special Variable	Description
\$0	Name of the script
\$1, \$2, ...	Positional arguments (first, second, etc.)
\$#	Number of arguments passed
\$*	All arguments as a single string
\$@	All arguments separately quoted
"\$@"	Each argument is treated as a separate entity
"\$*"	All arguments are treated as a single string
\$\$	Process ID of the script
\$?	Exit status of the last command

Example: Basic Argument Handling

Script: args.sh

```
#!/bin/bash
echo "Script Name: $0"
echo "First Argument: $1"
echo "Second Argument: $2"
echo "All Arguments: $@"
echo "Number of Arguments: $#"
```

Run the Script

```
bash args.sh Hello World
```

Output:

```
Script Name: args.sh
First Argument: Hello
Second Argument: World
All Arguments: Hello World
Number of Arguments: 2
```

Handling All Arguments

Example: Looping Through Arguments

```
#!/bin/bash
echo "List of Arguments:"
for arg in "$@"; do
    echo "$arg"
done
```

Run the Script

```
bash args.sh Apple Orange Banana
```

Output:

```
List of Arguments:
Apple
Orange
Banana
```

Checking If Arguments Are Passed

```
#!/bin/bash
if [ $# -eq 0 ]; then
    echo "No arguments provided."
    exit 1
fi
echo "Arguments provided."
```

Default Values for Arguments

```
#!/bin/bash
name=${1:-Guest}
echo "Hello, $name!"
```

Run the Script

```
bash args.sh
```

Output:

Hello, Guest!

(If no argument is provided, Guest is used as the default.)

Positional Parameters in Unix

Positional parameters in Unix are special variables used in shell scripts to pass and access arguments.

Basic Positional Parameters

Script: positional.sh

```
#!/bin/bash
echo "Script Name: $0"
echo "First Argument: $1"
echo "Second Argument: $2"
echo "Total Arguments: $#"
```

```
echo "All Arguments: $@"
```

Run the Script

```
bash positional.sh Hello World
```

Output:

```
Script Name: positional.sh
First Argument: Hello
Second Argument: World
Total Arguments: 2
All Arguments: Hello World
```

test Command in Unix

The test command in Unix is used to evaluate **conditional expressions**. It checks for file types, string comparisons, and numeric comparisons, returning an **exit status**:

- 0 (True)
- 1 (False)

Syntax:

```
test EXPRESSION      or
```

```
[ EXPRESSION ]
```

Note: The square brackets [...] are an alias for test but require spaces around them.

1. File Tests**Check if a File Exists**

test -e filename

OR

[-e filename]

Example:

```
if [ -e myfile.txt ]; then
    echo "File exists"
else
    echo "File does not exist"
fi
```

File Test Options

Test	Description
-e file	Check if the file exists
-f file	Check if it is a regular file
-d file	Check if it is a directory
-r file	Check if the file is readable
-w file	Check if the file is writable
-x file	Check if the file is executable
-s file	Check if the file is not empty
-L file	Check if the file is a symbolic link

Example: Check if a Directory Exists

```
if [ -d /home/user ]; then

    echo "Directory exists"
else
    echo "Directory does not exist"
fi
```

2. String test

Check if a String is Empty

```
if [ -z "$var" ]; then
    echo "String is empty"
fi
```

Check if a String is Non-Empty

```
if [ -n "$var" ]; then
    echo "String is not empty"
fi
```

Compare Two Strings

Test	Description
"\$a" = "\$b"	Strings are equal
"\$a" != "\$b"	Strings are not equal
-z "\$a"	String is empty
-n "\$a"	String is not empty

Example

```
str1="hello"
str2="world"
if [ "$str1" != "$str2" ]; then
    echo "Strings are different"
fi
```

3. Numeric test

Operator	Meaning
-eq	Equal to
-ne	Not equal to
-gt	Greater than
-ge	Greater than or equal to
-lt	Less than
-le	Less than or equal to

Example:

```
num1=10
num2=20
if [ $num1 -lt $num2 ]; then
    echo "$num1 is less than $num2"
fi
```

4. Using test in Scripts

Example: Check if a Number is Even

```
#!/bin/bash
num=$1
if [ $((num % 2)) -eq 0 ]; then
    echo "$num is even"
else
    echo "$num is odd"
fi
```

Run:

```
bash test_script.sh 10
```

Output:

```
10 is even
```

Expr command:

This command is used to perform mathematical calculations.

Syntax: ``expr operand1 operator operand2 ``

Where (```) this symbol is known as grep symbol. There is always must a space between symbol and the expr command.

Ex: clear echo “enter 2 numbers” read a b

```
echo “sum of 2 numbers is `expr $a + $b` “
echo “sub of 2 numbers is `expr $a - $b` “
echo “product of 2 numbers is `expr $a \* $b` “
echo “quotient of 2 numbers is `expr $a / $b` “
```

Note: In unix multiplication purpose we use the symbol of “`\*`” because only `*` is wild card character.

Control statements

The ability to control the flow of execution of program is known as control statements. The different types of control structures are

- Sequence control structure
- Selection control structure
- Looping control structure

Selection control structure

This type of instruction allows the shell script to be executed depending on the condition. There are mainly 4 types of decision making instructions. They are

- if-then-fi statement
- if-then-else-fi statement
- if-then-elif-then-else-fi statement
- case-esac statement

if-then-fi statement:

in this first check the condition. That condition is true then only the if block statements will be executed otherwise the cursor transfer outside the if condition.

Syntax:

```
if [ condition ]  
then  
statements  
fi
```

ex:

```
if [ $a -gt 18 ]  
then  
echo "eligible for vote"  
fi
```

if-then-else-fi statement:

in this first check the condition. That condition is true then only the if block statements will be executed otherwise the else block statements will be executed.

Syntax:

```
if [ condition ]  
then  
statements  
else  
statements  
fi
```

ex:

```
if [ $a -gt $b ]  
then  
echo "a is larger than b"  
else  
echo "b is larger than a"  
fi
```


if-then-elif-then-else-fi statement:

in this first check the condition. That condition is true then only the if block statements will be executed otherwise the cursor checks the next condition then the second condition will be true then inside that statements will be executed and so on. If any conditions were not true then the else block statements will be executed.

Syntax:

```
if [ condition 1 ]
then
statements
elif [ condition 2]
then
statements
else
statements
fi
```

ex:

```
if [ $a -gt $b -a $a -gt $c]
then
echo "a is larger"
elif [ $b -gt $c ]
then
echo " b is larger"
else
echo "c is larger"
fi
```

Case-esac statement:

Syntax:

```
case $variable in
[match1] (statements ; ;
[match2] statements ; ;
[match3] statements ; ;
:
:
:
*) statements ; ;
esac
```

- here match1,match2 etc are the case labels.
- When a case statement is evaluated the value of variable is matched in any one of the choices.
- When a match is found then shell executes that corresponding match statements.
- The pair of semicolon at the end of every choices. It identifies break.
- *) indicates default class. Ex: clear echo "enter a character"

Looping Control statements

In this all statements are executed repeatedly again and again as long as condition is true. This is also known as repetition or iteration.

Shell allows various types of looping. They are

- While loop
- Until loop
- For loop

While loop:

This is the pretested loop or entry controlled loop. In this first check the condition, if that condition was true then control enter inside the loop otherwise control transferred outside the loop.

Syntax:

```
while [ condition ]  
  
do  
  
Statements  
  
done
```

ex:

```
while [ i -le 10 ]  
  
do  
  
echo "$i"  
    i=`expr $i + 1`  
  
done
```

until loop:

This is also pretested loop or entry controlled loop. In this first check the condition, if that condition was false then control enter inside the loop otherwise control transferred outside the loop.

Syntax:

```
until [ condition ]  
  
do  
  
Statements  
  
done
```

ex:

```
until [ i -ge 10 ]  
  
do  
  
echo "$i"  
    i=`expr $i + 1`  
  
done
```

for loop:

This is a fixed execution loop. This loop is allow to execute list of statements certain period of time.

Syntax:

for variable in value1 value2 value3

..... value n do

statements

done

ex:

for I in 1 2 3 4 5

do echo \$

i=`expr \$i + 1 `

done

Jump Control Statements in Unix (Shell Scripting)

Jump control statements in Unix allow controlling the flow of execution by **skipping, repeating, or terminating loops** and conditional statements.

Types of Jump Control Statements

1. **break** → Exits a loop immediately.
2. **continue** → Skips the current iteration and moves to the next one.
3. **exit** → Terminates the script execution.

1. break Statement (Exit a Loop):

The break statement **terminates a loop** when a certain condition is met.

Example: Stop Loop on Condition

```
#!/bin/bash
for num in 1 2 3 4 5; do
    if [ "$num" -eq 3 ]; then
        echo "Breaking loop at $num"
        break
    fi
    echo "Number: $num"
done
```

Output:

Number: 1

Number: 2

Breaking loop at 3

Note: The loop stops when num == 3.

2. continue Statement (Skip Iteration):

The continue statement **skips the current iteration** and moves to the next one.

Example: Skip Even Numbers

```
#!/bin/bash
for num in 1 2 3 4 5; do
    if [ $((num % 2)) -eq 0 ]; then
        continue # Skip even numbers
    fi
    echo "Odd Number: $num"
done
```

Output:

```
Odd Number: 1
Odd Number: 3
Odd Number: 5
```

Note: The loop **skips even numbers** and continues with the next iteration.

3. exit Statement (Terminate Script)

The exit statement **stops script execution immediately**.

Example: Exit if File Not Found

```
#!/bin/bash
if [ ! -f "important.txt" ]; then
    echo "File not found! Exiting..."
    exit 1 # Exit script with error status 1
fi
echo "Processing file..."
```

Output if File is missing:

```
File not found! Exiting...
```

Note: The script stops execution if important.txt is missing.